



VNiVERSIDAD
D SALAMANCA

Fundamentals and Introduction to Machine Learning

Content for students of Advanced Programming on the Computer Engineering degree from the University of Salamanca

Diego M. Jiménez-Bravo

Universidad de Salamanca

CONTENTS

I	Section 1 - Fundamentals	3
1	Fundamentals	5
1.1	History and Evolution of Machine Learning	6
1.2	Fundamental Concepts	9
II	Section 2 - Supervised Learning	15
2	Supervised Learning	17
2.1	Regression	18
2.2	Classification	24
III	Section 3 - Unsupervised Learning	35
3	Unsupervised Learning	37
3.1	Clustering	38
3.2	Dimensionality Reduction	51
3.3	Anomaly Detection	60
	Bibliography	67
	Bibliography	69
	Course reference books	69
	Cited articles and studies	69
	About this book	71
	About	73
	How this book is made	73
	About the author	73
	Credits and licenses	75
	Project credits	75
	Acknowledgements	75
	Book license	75
	External resources	75
	Code and tools	75

How to Cite	77
Recommended Citation (APA 7th edition)	77
BibTeX	77
Author	77
DOI	77

Welcome to the course **Fundamentals and Introduction to Machine Learning** designed for students of the Faculties of Sciences and Chemical Sciences at the University of Salamanca.

Purpose of the Book

This book guides you on a journey from classical statistical concepts to modern deep learning models. The goal is for you to **understand how machines learn from data** and for you to be able to build, train, and evaluate models that solve real-world problems.

Content Structure

The book is divided into three main sections:

Section 1: Fundamentals

Understand the paradigm shift: from fixed-rule programming to data-driven learning.

- Historical origins of AI and machine learning
- Fundamental concepts: data, features, labels
- Evaluation metrics and cross-validation

Section 2: Supervised Learning

Learn to model continuous variables (regression) and categorical variables (classification).

- Linear and polynomial regression
- Linear and non-linear classifiers
- Neural networks

Section 3: Unsupervised Learning

Discover hidden patterns in unlabeled data.

- Clustering (grouping)
- Dimensionality reduction
- Anomaly detection

Who Is This For?

This course is designed for:

- Undergraduate students in Science, Physics, Statistics, Computer Science, or Engineering
- Anyone interested in machine learning and data analysis
- Professionals looking to update their skills

Prerequisites

While the book is self-contained, it helps to have:

- Basic Python knowledge or willingness to learn
- Familiarity with linear algebra and calculus (but not required)
- Analytical thinking and patience with mathematical concepts

How to Use This Book

Each chapter includes:

- **Theoretical concepts** explained clearly
- **Practical examples** with executable Java code
- **Visualizations** to better understand results

It's recommended to follow the established order, though you can consult specific sections as needed.

Let's begin! Go to Section 1: Fundamentals to start your machine learning journey.

Part I

Section 1 - Fundamentals

FUNDAMENTALS

Welcome to the first section of the course. Here we will understand the basic principles of machine learning and how it differs from the traditional programming approach.

Why Learn Machine Learning?

In traditional programming, a programmer explicitly writes all the rules that a program must follow:

```
if temperature > 30:
    turn_on_air_conditioning()
if temperature < 15:
    turn_on_heating()
```

But many real-world problems are too complex to describe with fixed rules. For example:

- How do we automatically distinguish a cat from a dog in a photograph?
- How do we predict if a bank transaction is fraudulent?
- How do we classify emails as spam or legitimate?

Machine learning inverts the paradigm: instead of writing the rules, we teach the program to *learn* the rules from examples.

What Will You Find in This Section?

1. **History and Evolution:** We'll learn how AI was born and evolved to reach the modern models we use today.
2. **Fundamental Concepts:** We'll learn the essential technical vocabulary for working with machine learning models.

These foundations are critical: without a solid understanding of these concepts, it will be difficult to understand the more advanced techniques in the later sections.

Let's start: Go to Chapter 1.1 to explore the fascinating history of machine learning.

1.1 History and Evolution of Machine Learning

1.1.1 Introduction

Understanding the history of machine learning helps us understand why certain approaches work better than others and why current research focuses on certain directions.

As shown in Fig. 1.1, the path traveled has not been linear, but rather a succession of mathematical breakthroughs, enthusiasm, funding winters, and resurgences.

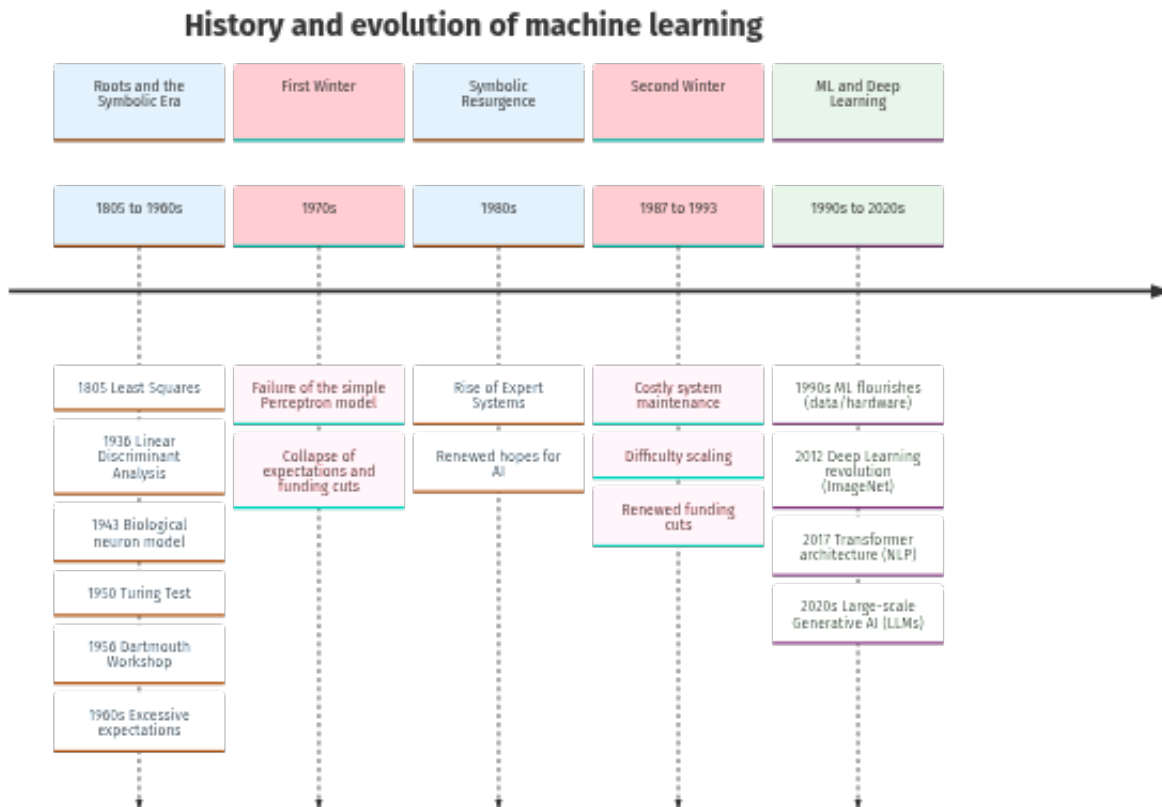


Figure 1.1: Timeline of the history and evolution of machine learning.

1.1.2 Definition and the programming paradigm shift

Artificial Intelligence is concisely defined as the effort to automate intellectual tasks normally performed by humans. It is a vast field that encompasses both machine learning and deep learning, but that also includes methods such as symbolic AI, which dominated the field from the 1950s until the late 1980s.

The most revolutionary aspect of machine learning is that it represents a **new programming paradigm**. In classical programming, a human writes rules (algorithms) that process data to produce answers. In machine learning, the process is reversed: the machine analyzes the input data and the expected answers to statistically infer what the rules should be. For this learning to occur, **three key elements** are required: input data points, examples of the expected output (labels), and a way to measure whether the algorithm is doing a good job of adjusting the system (a loss function).

1.1.3 Mathematical roots and pioneers (1943-1950)

Although the term is modern, many statistical learning concepts were developed decades earlier. In the early 19th century, Legendre and Gauss published on the **method of least squares** [Gau09, Leg05], which is the earliest form of linear regression. In 1936, Fisher proposed **linear discriminant analysis** [Fis36] for predicting qualitative values.

In the computational domain, the field began to take shape in the 1940s. In 1943, Warren McCulloch and Walter Pitts presented the **first computational model of biological neurons** [MP43]. Later, in 1950, Alan Turing published his foundational paper “Computing Machinery and Intelligence” [Tur50], where he introduced the **Turing Test** as a conceptual tool to discuss the nature of cognition in machines and the possibility that they could emulate human intelligence.



Figure1.2: Alan Turing in 1951, author of the paper that introduced the Turing Test. Source: photograph by Elliott & Fry (1951), Computer History Museum — public domain (Wikimedia Commons, published before 1956).

1.1.4 Formal birth and the symbolic era (1956-1980)

AI as a research field formally crystallized in 1956, when John McCarthy organized the **Dartmouth workshop** [MMRS06]. McCarthy proposed the conjecture that every aspect of learning or intelligence could be described with such precision that a machine could simulate it. For decades, experts believed that human-level intelligence would be achieved by manually coding a massive set of explicit logical rules. This approach, known as **symbolic AI**, reached its peak with **expert systems** in the 1980s.

1.1.5 The AI winters and the resurgence of machine learning

The history of AI has not been linear; instead, it has gone through cycles of extreme optimism followed by disillusionment and funding cuts, known as “AI winters.”

- **First winter:** occurred in the 1970s, following the failure of early symbolic AI’s expectations and of simple models such as the **Perceptron** [Ros58], which could not solve non-linear problems.
- **Second winter:** happened in the early 1990s, when expert systems turned out to be expensive to maintain, difficult to scale, and limited in scope.

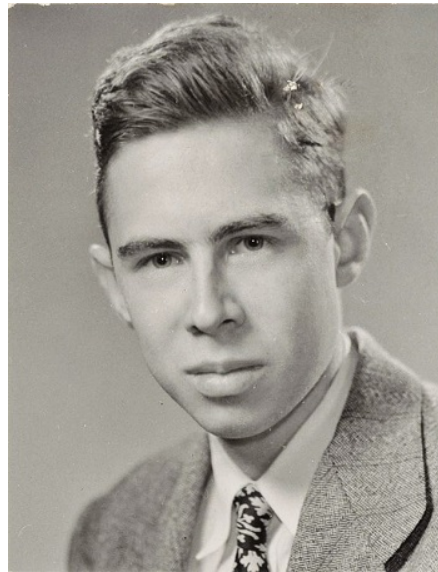


Figure1.3: Frank Rosenblatt (c. 1950), creator of the Perceptron in 1958. Source: Heinz Nixdorf MuseumsForum — Wikimedia Commons, CC BY-SA 4.0 license.

Machine Learning truly began to flourish in the 1990s, driven by the availability of faster hardware and massive datasets. Unlike symbolic AI, which was well suited to logical problems such as chess, ML made it possible to tackle “fuzzy” problems such as speech recognition or image classification.

1.1.6 From Deep Learning to the LLM era (2010-present)

Deep Learning is a specialized branch of ML that emphasizes learning successive layered representations. These layers, structured into neural networks, act as a multi-stage information distillation process where the data becomes increasingly useful for a specific task. The turning point for Deep Learning occurred between 2011 and 2015, highlighted by Hinton’s group winning the 2012 **ImageNet** challenge [KSH12].

During this era, the **MNIST** dataset (handwritten digits 0-9) became the quintessential benchmark for comparing classification algorithms, from early statistical classifiers to deep convolutional networks.

Starting in 2017, the **Transformer architecture** [VSP+17], which uses an attention mechanism to process sequences without recurrent layers, unleashed a revolution in natural language processing. This enabled the development of **foundation models** trained through **self-supervised learning** on enormous amounts of internet data. The current era, marked by **Generative AI** (such as ChatGPT and Gemini), has pushed the technology to a scale of billions of parameters, allowing it not only to classify but also to generate creative and functional content.

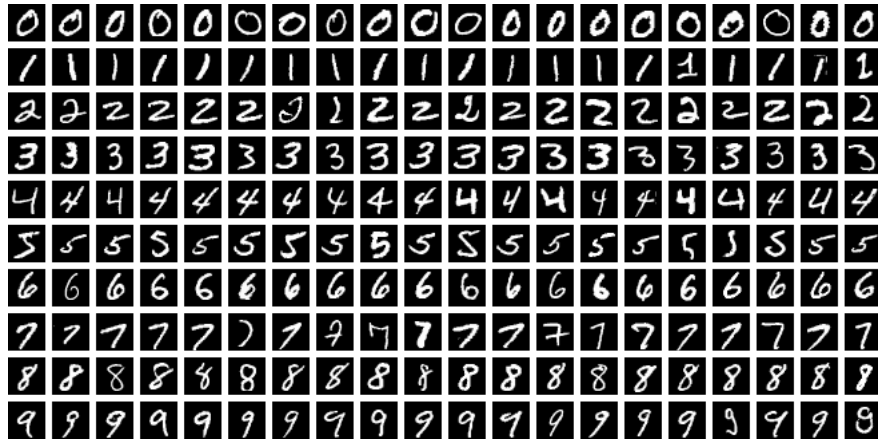


Figure 1.4: Samples from the MNIST dataset: different handwritten variants of each digit (0-9). Source: Suvanjanprasai (2024) — Wikimedia Commons, CC BY-SA 4.0 license.

1.1.7 Summary

- Machine learning was born as a response to the question: can machines learn?
- It has gone through cycles of hype and disillusionment.
- Recent advances stem from more data, better hardware, and innovative algorithms.
- Deep learning and LLMs represent the current state of the art.

Next: in Chapter 1.2, we will learn the fundamental concepts you need to work with any machine learning model.

1.2 Fundamental Concepts

1.2.1 Introduction

To work effectively with machine learning models, we need to master certain terminology and key concepts. This chapter builds the vocabulary we will use throughout the course.

1.2.2 Data, features, and labels

The success of any ML system depends on how information is represented.

Sample	Feature	Feature	Feature	Label
	1	2	3	
Example	Age	Weight	Height	Type
1	25	70	175	Cat
2	8	4	90	Cat
3	35	80	180	Dog

- **Sample:** the basic unit of information, also called a data point, instance, or example. In a tabular dataset, each row represents a single sample.

- **Attributes and features:** samples are characterized by their features (attributes), which are quantitative or qualitative variables measuring different aspects of the instance. Technically, an “attribute” is a data type (e.g., “age”), while a “feature” usually refers to the attribute together with its specific value (e.g., “age = 25”). In deep learning, all inputs are vectorized so they can be processed as points in a geometric space.
- **Label or target:** in supervised learning, each sample has an associated correct answer called a label or target. The set of labels for an entire dataset constitutes the “**ground-truth.**” In classification problems, the goal is to predict a category (class), while in regression the goal is a continuous numeric or scalar value.

```
# Features: can be numbers, categories, etc.
features = [
    "square_meters",
    "number_of_rooms",
    "location",
    "year_built"
]

# Label: what we want to predict
label = "price"
```

1.2.3 The tension between optimization and generalization: overfitting and underfitting

The fundamental problem of ML is the tension between fitting the model to known data and its ability to perform on new data.

- **Optimization:** the process of adjusting a model’s parameters (weights) to obtain the best possible performance on the training data.
- **Generalization:** refers to how well the trained model performs on data it has never seen before.
- **Overfitting:** occurs when a model is too complex relative to the amount and noise of the data. The model “memorizes” training noise or random patterns that do not exist in the real data, resulting in low training loss but very high validation loss.
- **Underfitting:** happens when the model is too simple to capture the underlying structure of the data. In this state, both training and validation error are high.
- **Bias-variance trade-off:** generalization error is split into:
 - **Bias:** error from incorrect assumptions (e.g., assuming a linear relationship when it is actually quadratic); high bias causes underfitting.
 - **Variance:** error from excessive sensitivity to small variations in the training data; high variance causes overfitting.

Fig. 1.5 illustrates the three scenarios by fitting different models to the same data: a line that is too simple (left), a moderate-degree polynomial that follows the true trend (center), and a very high-degree polynomial that memorizes every point, including the noise (right).

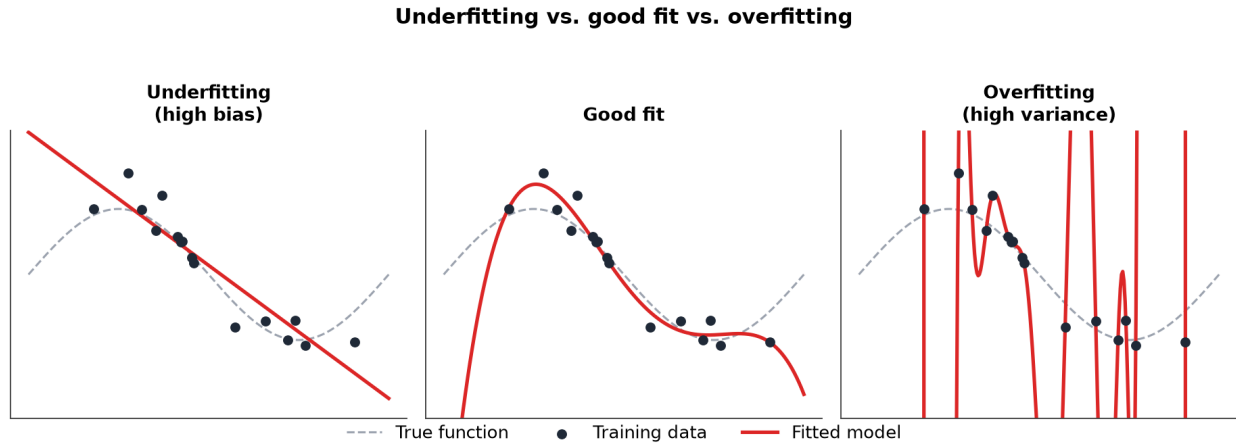


Figure1.5: Underfitting (high bias) vs. good fit vs. overfitting (high variance).

1.2.4 Evaluation protocols: cross-validation

To measure generalization reliably, it is not enough to evaluate the model on the same data it was trained on.

- **Data splitting:** standard practice is to split the data into three sets: training (to learn the weights), validation (to choose hyperparameters and avoid information “leakage”), and test (for a final, unbiased evaluation).
- **K -fold cross-validation:** when data is scarce, a simple split can be unrepresentative. This method splits the data into K partitions (typically 5 or 10). The model is trained K times; each iteration uses a different partition for validation and the remaining $K - 1$ for training. The final score is the average of the K results obtained, which reduces the variance of the evaluation.
- **Stratification:** in classification, it is vital that each “fold” preserves the same class proportions as the original dataset to avoid bias, a process known as stratified K -fold.

Fig. 1.6 shows an example with $K = 5$: in each row (each “fold”), a different block of data acts as the validation set (in red) while the rest is used for training (in blue).

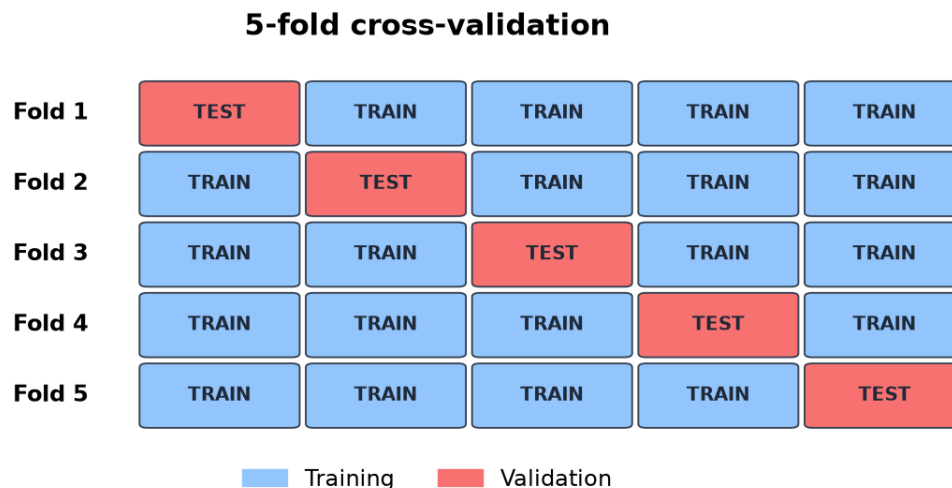


Figure1.6: K -fold cross-validation with $K = 5$: each block of data acts once as the validation set.

1.2.5 Performance metrics

Metrics allow us to quantify the model's success and guide technical decisions.

- **Confusion matrix:** a two-dimensional tool used to evaluate a classifier's performance in detail. Its structure is a table where each row represents the true class (ground-truth) and each column represents the class predicted by the model. In a binary classification problem (two classes), the matrix contains four fundamental values that break down the system's hits and misses:
 - **True Positives (TP):** instances the model correctly classified as positive.
 - **True Negatives (TN):** instances the model correctly classified as negative.
 - **False Positives (FP):** occurs when the model incorrectly predicts that an instance is positive when it is actually negative. This type of error is also known as a Type I error or "false alarm."
 - **False Negatives (FN):** occurs when the model incorrectly predicts that an instance is negative when it is actually positive. Also called a Type II error or "miss."

Fig. 1.7 visually summarizes these four values: the main diagonal (green) represents the model's hits, while the secondary diagonal (red) represents its two types of error.

		Prediction	
		Positive	Negative
Actual class	Positive	TP True Positive	FN False Negative
	Negative	FP False Positive	TN True Negative

Figure 1.7: Confusion matrix: hits (green) and Type I and Type II errors (red).

- **Accuracy:** the fraction of correct predictions over the total. Can be misleading if classes are imbalanced.

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

- **Precision:** the model's ability to avoid labeling a negative sample as positive. $\text{Precision} = \frac{TP}{TP+FP}$
- **Recall:** the model's ability to find all the actual positive samples ($TP/(TP+FN)$). $\text{Recall} = \frac{TP}{TP+FN}$
- **F1-score:** the harmonic mean of Precision and Recall; useful when seeking a balance between the two, penalizing extreme values. $F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$
- **AUC-ROC:** the area under the receiver operating characteristic curve represents the probability that the model ranks a random positive sample above a random negative one. An AUC of 1.0 is perfect, while 0.5 is equivalent to random classification.

1.2.6 The ML pipeline: the universal workflow

Developing a Machine Learning project follows a universal blueprint that ensures the system not only learns from the data but is also able to generalize its results in production environments. Fig. 1.8 summarizes the 10 stages and their cyclical nature: hyperparameter tuning usually requires retraining the model several times before the final evaluation.

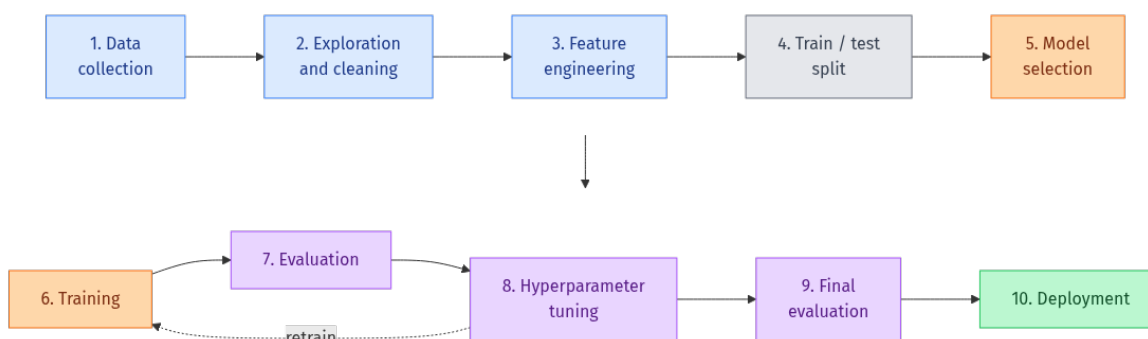


Figure 1.8: The Machine Learning pipeline: 10 stages from data collection to deployment.

Below, the stages of the ML pipeline are defined, integrating the theoretical and technical foundations covered so far:

1. **Data collection:** this is the most arduous and costly phase, requiring an understanding of the problem domain and business objectives. It consists of obtaining representative samples from the real environment, which often involves manual annotation or labeling by human experts to generate the ground-truth needed for supervised learning.
2. **Exploration and cleaning:** it is bad practice to treat the dataset as a black box; the data distribution should be visualized using histograms or maps to detect anomalies and outliers. Cleaning includes handling missing values (either by removing records or imputing averages) and detecting labeling errors to prevent noise from degrading model performance.
3. **Feature engineering:** applying human knowledge to perform non-learned transformations that make the algorithm's task easier, such as normalizing numeric scales or standardizing text to remove encoding differences. Although Deep Learning automates much of this process, good feature engineering makes it possible to solve problems with far less data and computational resources.
4. **Train/test split:** to evaluate generalization ability, the data must be strictly split into training, validation, and test sets. Random shuffling is critical to ensure representativeness, except for time series, where the test set must be chronologically later than the training set to avoid leaking information from the future.
5. **Model selection:** at this stage, the appropriate architecture priors are chosen for the task, such as convolutional networks for images or Transformers for natural language. The first technical goal is to develop a simple model that beats a common-sense baseline, demonstrating that there is an exploitable statistical pattern in the data.

6. **Training:** training is an iterative process called the training loop, where the model's weights are randomly initialized and gradually adjusted. Through the forward pass, loss computation, and gradient backpropagation, the system optimizes its internal parameters to minimize error on the training data.
7. **Evaluation:** the validation set is used to monitor the model's performance during training using metrics such as precision, recall, or AUC. This phase makes it possible to detect whether the model is underfitting or has started memorizing the noise in the training data.
8. **Hyperparameter tuning:** hyperparameters are external configurations (such as the number of layers or the learning rate) that are not learned via gradient descent but rather through systematic search. Techniques such as K -fold cross-validation or Bayesian optimization are used to find the configuration that maximizes generalization before the final evaluation.
9. **Final evaluation:** once the best model has been selected and its hyperparameters tuned, a single evaluation is performed on the test set, which must have remained in a "vault" without being consulted beforehand. If results here are significantly worse than in validation, it is a sign that overfitting occurred to the validation process or that the data lacked representativeness.
10. **Deployment:** the final model is exported for production use, whether as a web API, in mobile applications, or on embedded devices. The pipeline does not end here; it is imperative to implement constant monitoring, since real-world data usually degrades over time (concept drift), requiring periodic retraining cycles.

1.2.7 Summary

- **Sample:** a single point in the dataset.
- **Features:** input variables that describe a sample.
- **Label:** the output variable we want to predict.
- **Overfitting:** memorizing training data, poor on test data.
- **Underfitting:** model too simple, poor on both.
- **Cross-Validation:** a technique for evaluating performance without bias.
- **Precision:** ratio of correctly predicted positives.
- **Recall:** ratio of actual positives found.
- **ML Pipeline:** composed of 10 stages, from data collection to production deployment.

Next: now that you understand the fundamentals, we move on to Section 2: Supervised Learning, where you will learn to build practical models.

Part II

Section 2 - Supervised Learning

SUPERVISED LEARNING

Welcome to the second section of the course. Here we will learn to build models that learn from labeled examples.

What is supervised learning?

In supervised learning, we have:

- **Data:** input features (X).
- **Labels:** correct answers (y).
- **Goal:** learn the relationship $X \rightarrow y$.

It is like learning with a “supervisor” who tells you whether your answers are correct.

Two types of problems

1. Regression

We predict a **continuous value** (a real number):

- Predicting the price of a house.
- Predicting tomorrow’s temperature.
- Predicting rainfall amount.
- Predicting population growth.

2. Classification

We predict a **category** or **class**:

- Diagnosing whether a person has a disease (yes/no).
- Classifying emails as spam or not spam.
- Recognizing whether an image is a cat, dog, or bird.
- Predicting whether a customer will churn.

What Will You Find in This Section?

- **Regression:**
 - Linear regression: the simplest and most fundamental model.
 - Ordinary least squares: how a linear regression is trained.
 - Gradient descent: the most important optimization algorithm.
 - Polynomial regression and regularization: how to build more complex models.
 - **Classification:**
 - Logistic regression: binary classification.
 - Confusion matrix and metrics: how to evaluate classifiers.
 - Decision trees and Random Forests: interpretable models.
 - Support Vector Machines: optimal class separation.
 - Ensemble methods: combining multiple models.
 - Neural networks: an introduction to deep networks.
-

Let's start: go to Chapter 2.1 to learn about regression, the foundation of many machine learning models.

2.1 Regression

2.1.1 Introduction

Regression is the problem of predicting a **continuous value**. In other words, regression predicts a quantitative response or continuous scalar value Y from one or more **predictor variables** X .

2.1.2 Linear regression

Linear regression is a fundamental technique that assumes an approximately linear relationship between the predictors and the target variable.

- **Simple vs. multiple:** **simple linear regression** uses a single predictor to model $Y \approx \beta_0 + \beta_1 X$. **Multiple linear regression** extends this concept by assigning a specific coefficient or weight to each of the p available features: $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \epsilon$.
- **Model parameters:** the coefficients β_0 (intercept or bias) and $\beta_1 \dots \beta_p$ (weights or slopes) represent the knowledge the model has learned from the data. Technically, in a p -dimensional input space, this model represents a **hyperplane**.

Fig. 2.1 shows an example with a single predictor: the red line is the OLS fit and the gray vertical lines represent the residuals (the distance between each point and the prediction).

A classic example from the literature [JWHT13] is the *Advertising* dataset, where a line is fit to predict `Sales` from TV advertising spending (TV):

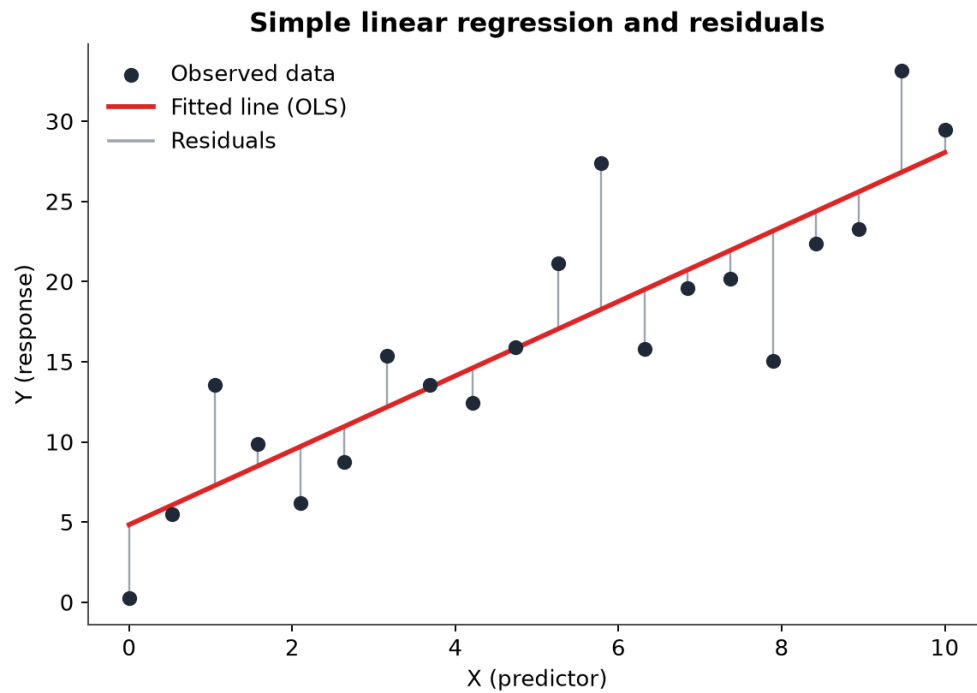


Figure2.1: Simple linear regression: fitted line and residuals.

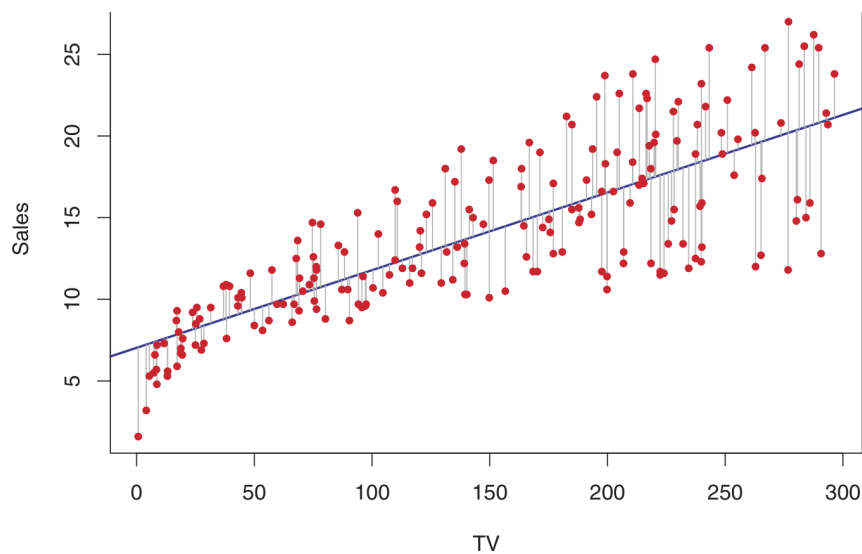


Figure2.2: Least squares fit of *Sales* on *TV* in the *Advertising* dataset. Source: James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*, Figure 3.1. Springer. Freely distributed for educational use (statlearning.com).

2.1.3 Ordinary least squares

The ordinary least squares (OLS) method is the standard approach for training linear models, seeking the parameters that best fit the training data.

- **Residuals and RSS:** a residual $e_i = y_i - \hat{y}_i$ is the difference between the actual value and the predicted one. The goal of OLS is to minimize the **sum of squared residuals** (RSS, Residual Sum of Squares): $RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$.
- **Analytical solution** (normal equations): unlike other complex models, the linear model has a unique, global solution that can be computed directly via matrix algebra: $\hat{\beta} = (X^T X)^{-1} X^T y$.
- **Geometric interpretation:** the prediction vector $\hat{y}$ is the **orthogonal projection** of the observation vector y onto the linear subspace spanned by the columns of the feature matrix X .

2.1.4 Gradient descent

For massive datasets that do not fit in memory, or models with thousands of parameters, solving the normal equation is computationally expensive. Gradient descent (GD) is the iterative optimization algorithm that powers modern machine learning.

- **Optimization mechanics:** the algorithm initializes the weights randomly and gradually adjusts them by measuring the local gradient of the loss function with respect to the parameters. It moves in the direction opposite the gradient (moving “down” the error curve) until it reaches a minimum.
- **Learning rate (η):** a critical hyperparameter that determines the step size at each iteration. A rate that is too high can cause divergence, while one that is too low will make training excessively slow.
- Variants:
 - **Batch GD:** uses the entire training set for each step.
 - **Stochastic GD (SGD):** uses a single random sample per step, making it much faster and able to handle online data streams.
 - **Mini-batch GD:** processes small groups of data (batches), balancing the stability of Batch GD with the efficiency of SGD.
- **Momentum:** a technique inspired by physics that helps avoid local minima and accelerates convergence by taking previous weight updates into account.

As shown in Fig. 2.3, the algorithm starts at an initial point and takes successive steps in the direction of steepest descent until it converges to the minimum of the loss function.

2.1.5 Polynomial regression and regularization

When data exhibits curvature, linear models can be extended through feature engineering.

- **Polynomial regression:** powers of the original predictors are added (e.g., X^2 , X^3) as new features. Although the model is non-linear with respect to the original inputs, it remains linear in its parameters, which allows OLS to be used for training.
- **The risk of overfitting:** increasing the polynomial degree increases the model’s flexibility, which can lead it to memorize the noise in the training data (overfitting) and fail to generalize.
- **Regularization techniques:** these add a penalty to the cost function for having large weights, forcing the model to be simpler.

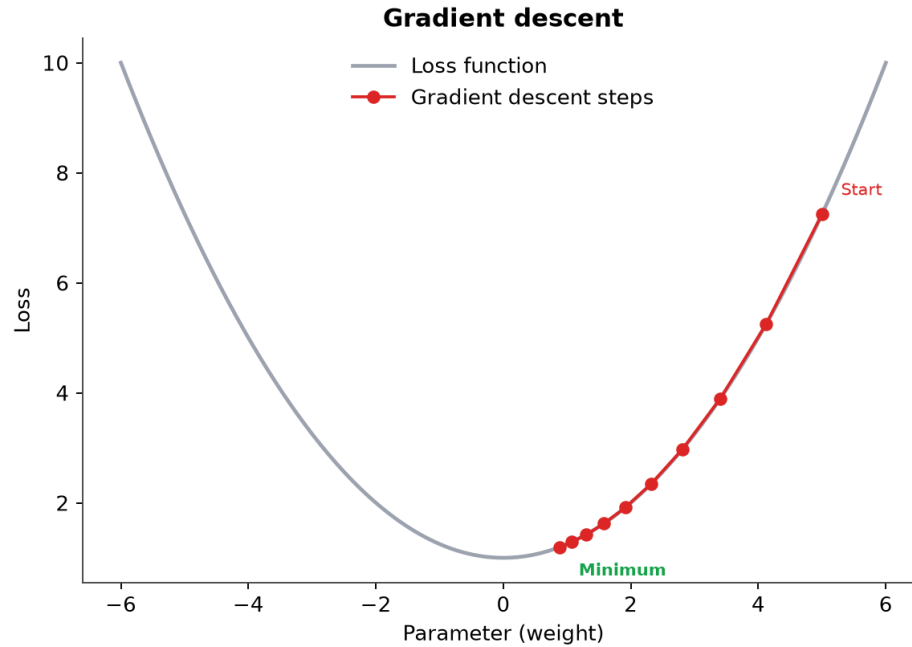


Figure 2.3: Gradient descent: iterative steps toward the minimum of the loss.

- **Ridge** (L2) [HK70]: adds a penalty proportional to the square of the weights. It shrinks coefficients toward zero but never eliminates them entirely; it works very well when there are many correlated predictors.
- **Lasso** (L1) [Tib96]: adds a penalty proportional to the absolute value of the weights. It has the unique property of forcing some coefficients to be exactly zero, automatically performing variable selection.
- **Elastic Net**: a combination of Ridge and Lasso that uses both types of penalty via a mixing ratio.

Fig. 2.4 compares how a model's coefficients evolve as the penalty λ increases: Ridge shrinks them smoothly without ever reaching zero, while Lasso drives them exactly to zero, performing variable selection.

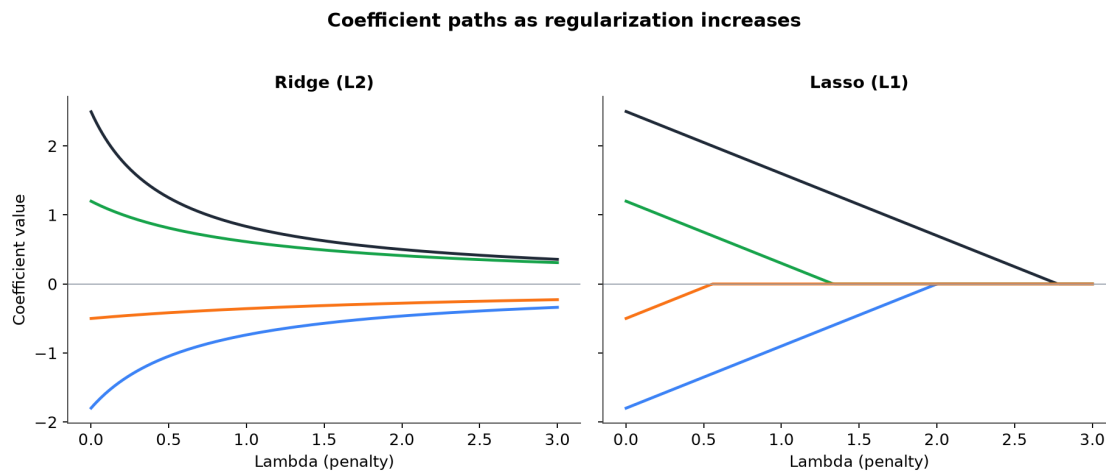


Figure 2.4: Coefficient paths: Ridge (smooth shrinkage) vs. Lasso (variable selection).

On real data, Fig. 2.5 shows how the standardized Ridge coefficients evolve for the *Credit* dataset, both as a function of λ (left) and of the relative coefficient norm (right).

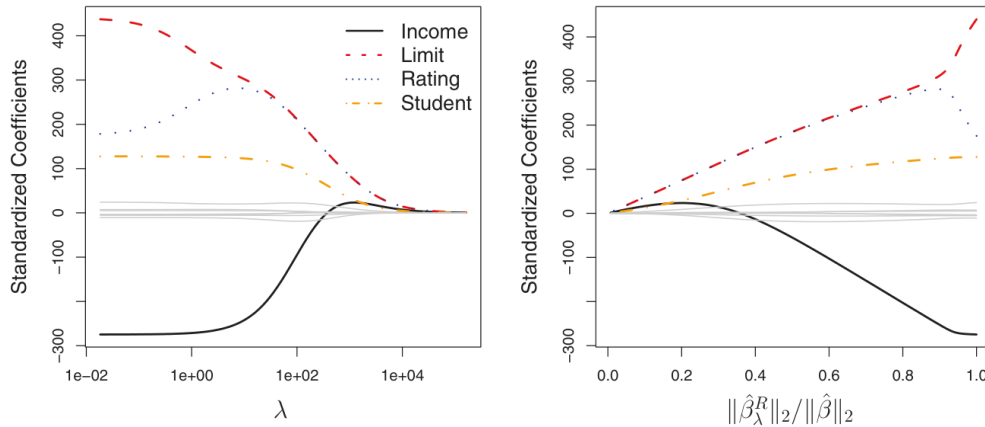


Figure 2.5: Standardized Ridge regression coefficients for the *Credit* dataset. Source: James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*, Figure 6.4. Springer. Freely distributed for educational use (statlearning.com).

2.1.6 Validation and diagnostics

Evaluating a regression model requires specific metrics and analysis of fit quality.

- Error metrics:
 - **MSE** (Mean Squared Error): the average of squared errors; strongly penalizes large deviations. $MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$
 - **RMSE** (Root MSE): the square root of MSE, which returns the error to the same units as the target variable. $RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$
 - **MAE** (Mean Absolute Error): the average of the absolute values of the errors; more robust to outliers. $MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$
- **Coefficient of determination (R²):** measures the proportion of the variance in Y that is explained by the model. A value of 1 indicates a perfect fit, while 0 indicates the model is no better than always predicting the mean.
- **Residual analysis:** plotting residuals against fitted values makes it possible to detect problems such as non-linearity. If the plot shows a pattern (such as a U-shaped curve), it indicates the linear model is insufficient.
- **Validation curves:** validation curves are fundamental graphical tools for diagnosing model behavior as a function of complexity.
 - **Bias-variance diagnosis:** by plotting training and validation error against model complexity (e.g., polynomial degree), we can identify:
 - * **High bias:** high error in both training and validation.
 - * **High variance:** very low training error but very high validation error; there is a significant gap between the two curves.
 - **One-standard-error rule:** in practice, it is common to choose the simplest model that falls within one standard deviation of the minimum error on the validation curve, to ensure parsimony.

2.1.7 Hands-on example in Java with SMILE

The concepts in this section can be put into practice with **SMILE** (Statistical Machine Intelligence and Learning Engine), a machine learning library for Java. The [programacion-avanzada-smile](#) repository from the Advanced Programming course includes the full example `Ejemplo04Regresion.java`, from which the following fragment is taken (comments translated into English).

The `viviendas.csv` (housing) dataset is synthetic: the price was generated from a known linear relationship plus Gaussian noise ($\text{precio} = 1.8 \cdot \text{superficie} + 12 \cdot \text{habitaciones} + 9 \cdot \text{banos} - 1.1 \cdot \text{antiguedad} - 6.5 \cdot \text{distancia_centro} + 55 + \varepsilon$), so you can check whether OLS recovers the true coefficients.

```
import smile.data.DataFrame;
import smile.data.formula.Formula;
import smile.io.Read;
import smile.regression.LASSO;
import smile.regression.LinearModel;
import smile.regression.OLS;
import smile.regression.RidgeRegression;
import smile.validation.CrossValidation;
import smile.validation.metric.R2;
import smile.validation.metric.RMSE;

DataFrame viviendas = Read.csv("data/viviendas.csv", "header=true");
Formula formula = Formula.lhs("precio"); // precio ~ all other columns

// Utiles.split shuffles the rows and splits them into train/test
Utiles.Particion particion = Utiles.split(viviendas, 0.75);
DataFrame train = particion.train();
DataFrame test = particion.test();

// 1. Ordinary least squares: exact solution in a single step
LinearModel ols = OLS.fit(formula, train);
System.out.println(ols); // coefficients, p-values, R2... (like summary() in R)
System.out.printf("Intercept: %.3f (true: 55)%n", ols.intercept());

// 2. Evaluation on the test set
double[] yTest = formula.y(test).toDoubleArray();
double[] prediccion = ols.predict(test);
System.out.printf("RMSE = %.3f%n", RMSE.of(yTest, prediccion));
System.out.printf("R2 = %.4f%n", R2.of(yTest, prediccion));

// 3. Regularization: Ridge shrinks the coefficients, LASSO can zero them out
LinearModel ridge = RidgeRegression.fit(formula, train, 0.1);
LinearModel lasso = LASSO.fit(formula, train, new LASSO.Options(0.5));
System.out.printf("Ridge RMSE = %.3f%n", RMSE.of(yTest, ridge.predict(test)));
System.out.printf("LASSO RMSE = %.3f%n", RMSE.of(yTest, lasso.predict(test)));

// 4. 10-fold cross-validation
var cv = CrossValidation.regression(10, formula, viviendas,
    (f, datos) -> OLS.fit(f, datos));
System.out.printf("OLS mean RMSE = %.3f | mean R2 = %.4f%n",
    cv.avg().rmse(), cv.avg().r2());
```

A few things worth noticing when you run it:

- `Formula.lhs("precio")` declares the target variable; every other column of the `DataFrame` is used as a predictor.
- `OLS.fit` solves the least-squares normal equations exactly, and the resulting model prints the estimated coeffi-

cients together with their standard error and p -value.

- The full example also compares OLS against Random Forest and Gradient Boosting: with data generated by a linear model, OLS beats the tree-based models, illustrating that “more complex” does not mean “better”.

To run it from the root of the repository: `mvn exec:java -Dexec.mainClass=es.usal.smile.Ejemplo04Regression`.

2.1.8 Summary

- **Linear Regression:** a simple yet powerful model.
 - **OLS:** an exact solution, but computationally expensive.
 - **Gradient Descent:** a general method that works for any model.
 - **Polynomial Regression:** captures non-linear relationships.
 - **Regularization:** prevents overfitting (Ridge for retention, Lasso for selection).
 - **Validation:** specific metrics for validating regression, together with a study of the training process.
-

Next: in the next chapter, we will extend these ideas to **Classification**, where we predict categories instead of continuous numbers.

2.2 Classification

2.2.1 Introduction

In classification, our goal is to predict **categories** (classes) rather than continuous values. To classify a sample or instance with a specific class, classification algorithms rely on the sample’s attributes. Some typical examples include:

- Email classification: spam vs. not-spam.
- Medical classification: sick vs. healthy.
- Species classification: cat vs. dog, etc.
- Etc.

2.2.2 Fundamentals and logistic regression

Unlike linear regression, which predicts continuous values, classification starts by estimating the probability of belonging to a category.

- **Model mechanics:** logistic regression uses the **sigmoid function** to transform a linear combination of the input features into a value between 0 and 1. The formula is defined as: $\sigma(t) = \frac{1}{1+e^{-t}}$

The input t of this function is called the **logit** (or log-odds), representing the unnormalized log-probabilities of the positive class.

- **Training and optimization:** the model is trained by minimizing a cost function called **cross-entropy** (or log loss), which penalizes predictions that are confident but wrong. Since this function is convex, gradient descent can be used to find the optimal weights iteratively.

- **Evaluation via confusion matrix:** a $K \times K$ table that breaks down the model's performance by comparing actual labels (rows) with predicted labels (columns). Four critical values arise from this:
 - **True Positives (TP)** and **True Negatives (TN)**: the model's correct predictions.
 - **False Positives (FP)**: Type I error or false alarm.
 - **False Negatives (FN)**: Type II error or miss.

Fig. 2.6 shows the characteristic “S” shape of the sigmoid function: very negative inputs are squashed toward 0 and very positive inputs toward 1, with the decision point at 0.5.

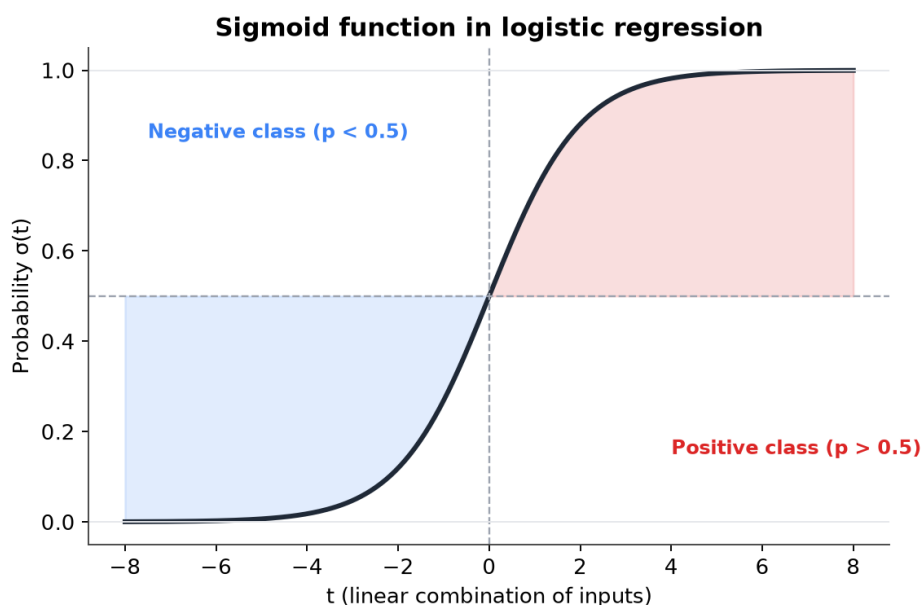


Figure2.6: Sigmoid function used in logistic regression.

Fig. 2.7 shows why this transformation is necessary: on the *Default* dataset, a linear regression fit (left) predicts absurd probabilities outside the $[0, 1]$ range, while logistic regression (right) always produces valid probabilities.

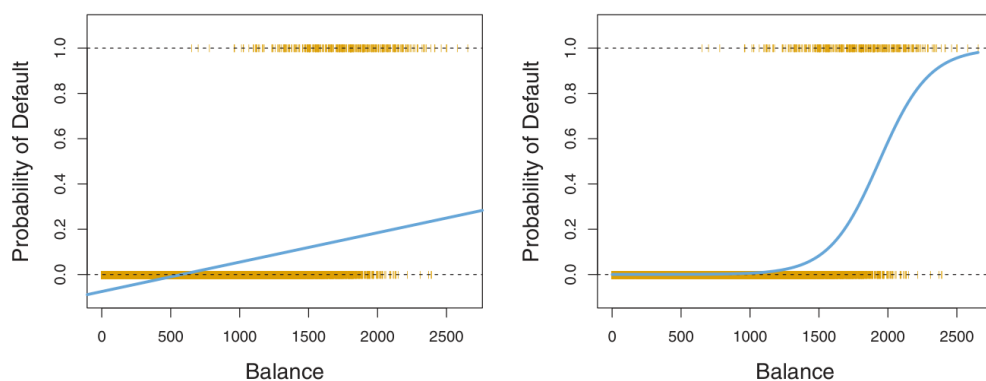


Figure2.7: Linear regression (left) vs. logistic regression (right) on the *Default* dataset. Source: James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*, Figure 4.2. Springer. Freely distributed for educational use (statlearning.com).

2.2.3 Classic classification algorithms

These methods stratify the feature space or search for geometric separation boundaries.

Decision Trees

Trees recursively partition the input space into high-dimensional “boxes” or rectangles.

- **Construction:** they follow a “divide and conquer” approach via the **CART algorithm**, which performs binary splits seeking to maximize node purity (reducing impurity metrics such as the Gini index or entropy).
- **Pruning:** to avoid overfitting, cost-complexity pruning is used, which removes unnecessary branches based on statistical tests or validation sets.

Fig. 2.8 illustrates how a tree divides the feature space into rectangular regions through successive axis-aligned cuts.

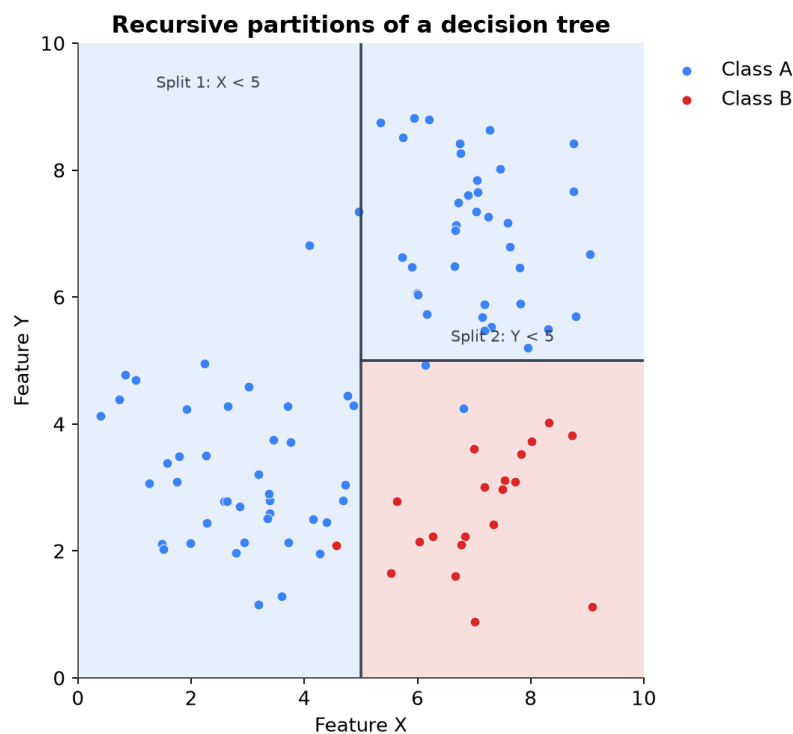


Figure 2.8: Recursive partitions of a decision tree over two features.

Random Forests

An ensemble of decision trees [Bre01] designed to reduce variance and improve robustness.

- **Bagging:** uses sampling with replacement (bootstrap) to train each tree on a different version of the data.
- **Decorrelation:** to ensure the trees are diverse, only a random subset of features is considered at each split.

Fig. 2.9 sketches the full process: several bootstrap samples are drawn from the original dataset, each trains a different tree (also using a random subset of features at each split), and the individual predictions are combined through majority voting (classification) or averaging (regression).

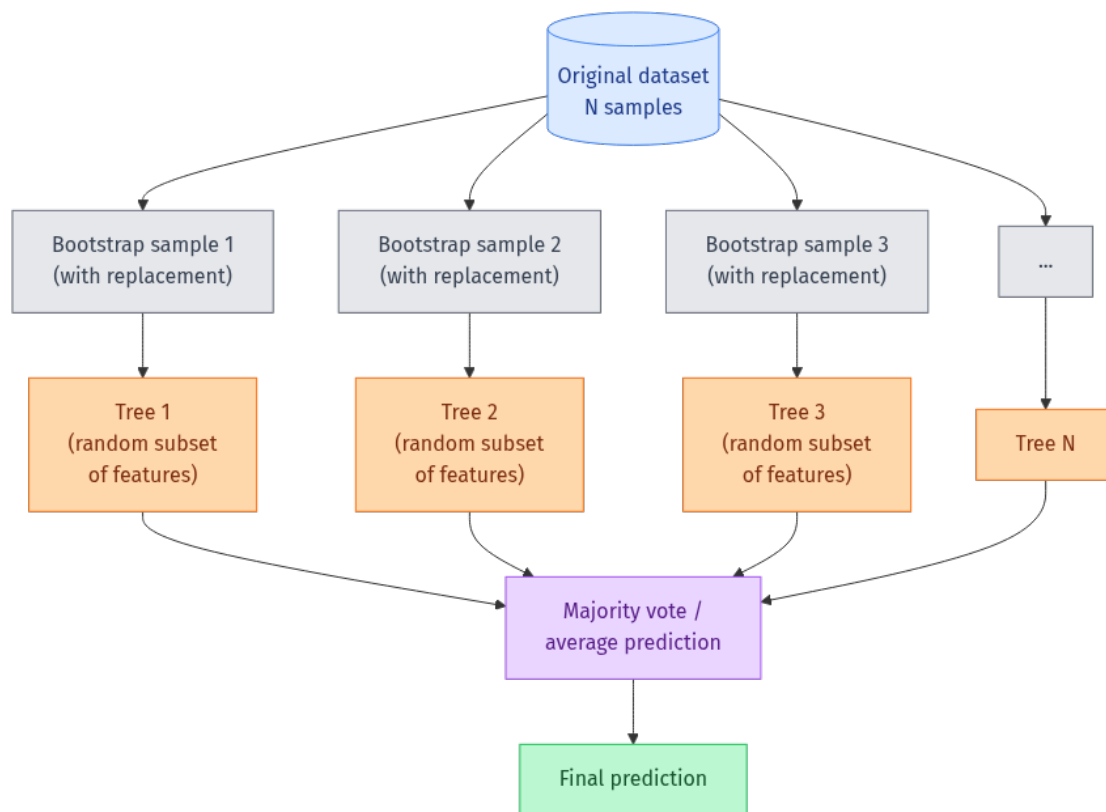


Figure2.9: Bagging: each tree is trained on a different bootstrap sample and predictions are combined by voting.

Support Vector Machines (SVM)

This model [CV95] seeks to find a **hyperplane** that separates the classes with the **maximum margin** possible.

- **Support vectors:** the decision boundary is determined solely by the samples closest to the hyperplane; moving other points does not change the model.
- **Soft margin and the kernel trick:** SVMs handle non-linearly separable data through slack variables and the kernel trick, which implicitly maps the data to higher-dimensional spaces to find complex separations.

Fig. 2.10 shows the optimal hyperplane (solid line), the maximum margin (gray band), and the support vectors (circled in green), which are the only points that determine the boundary's position.

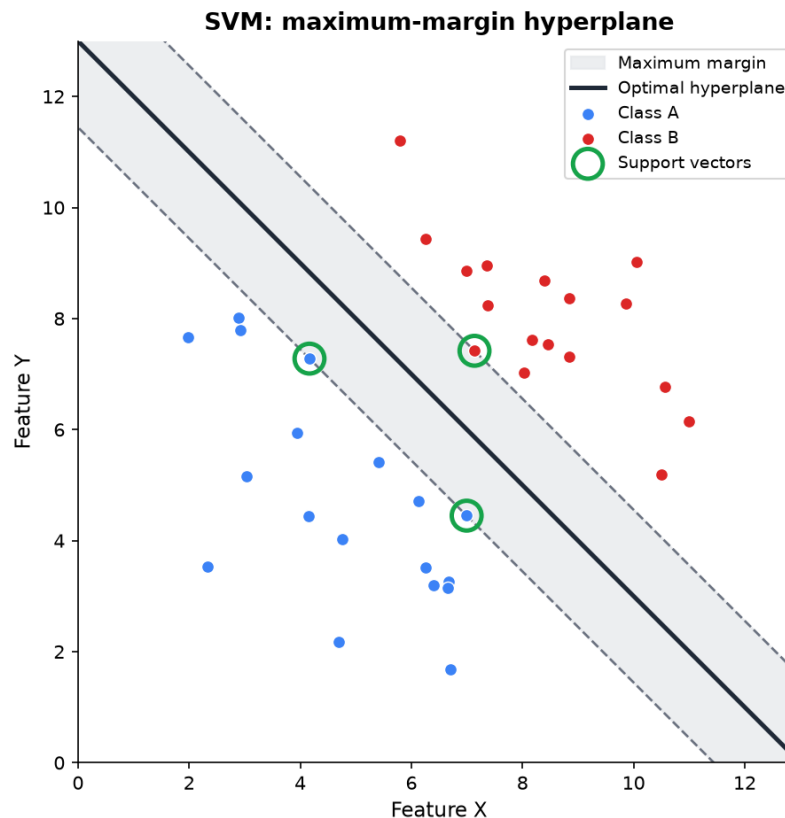


Figure 2.10: SVM: maximum-margin hyperplane and support vectors.

2.2.4 Advanced ensemble methods

These combine multiple models to obtain a prediction superior to that of any individual model.

- **Voting and stacking:** majority (hard) voting or averaging probabilities (soft voting) combines independent models. Stacking trains a meta-model that learns to combine the outputs of the base models.

Fig. 2.11 compares both approaches: in voting, the models are trained independently and their outputs are combined with a fixed rule (vote or average); in stacking, the base models' predictions become the input of a meta-model that learns how to combine them.

- **Boosting** [Fri01]: unlike the parallel training of random forests, boosting trains models sequentially, where each new predictor tries to correct the errors made by its predecessors. Modern examples include XGBoost and Gradient Boosting.

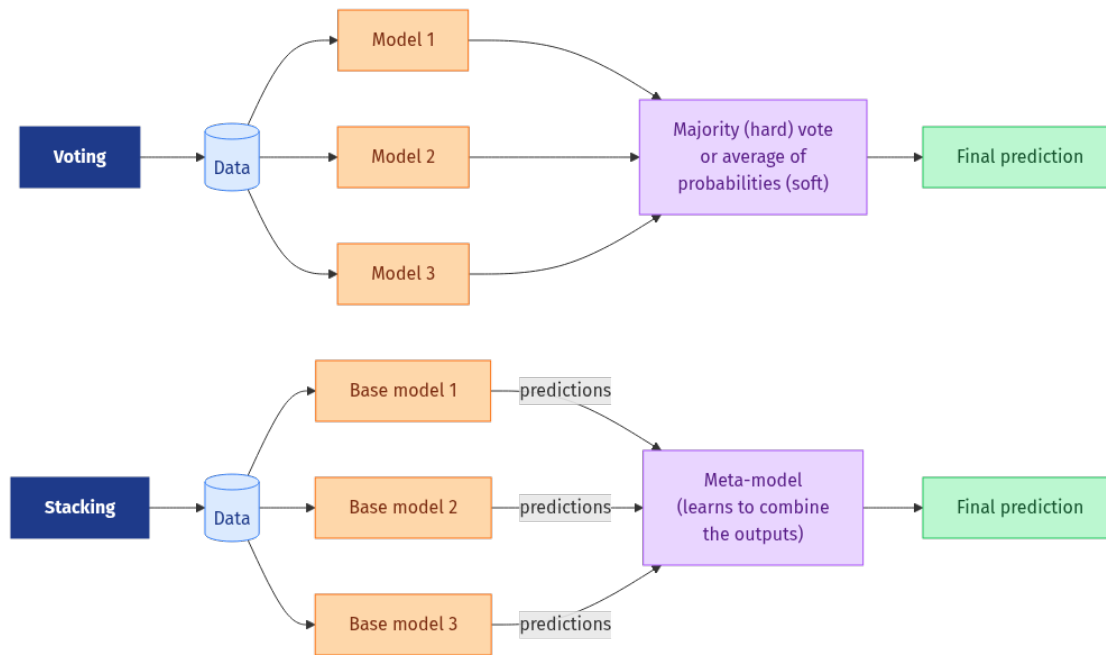


Figure 2.11: Voting versus stacking: a fixed combination rule versus a meta-model that learns to combine.

Fig. 2.12 illustrates boosting's sequential training: each weak model is trained on data reweighted according to the previous model's errors, giving more weight to misclassified instances, and the final prediction combines all models through a weighted sum.

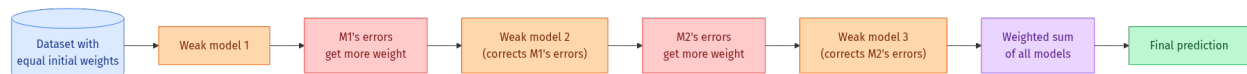


Figure 2.12: Boosting: models are trained sequentially, each one correcting the previous one's errors.

2.2.5 Introduction to neural networks

These represent learning through successive layers of filtering representations.

- **Architecture:** composed of an input layer, multiple hidden (densely connected) layers, and an output layer.
- **Activation functions:** introduce non-linearity to learn complex patterns. **ReLU** is the standard for hidden layers, while **Softmax** is used in the output layer for multi-class classification.
- **Backpropagation** [RHW86]: the central learning mechanism; it uses the chain rule from calculus to propagate the error backward from the output, adjusting the network's weights to reduce the total loss.

Fig. 2.13 sketches a fully-connected network with an input layer, a hidden layer, and an output layer: each connection represents a weight that gets adjusted during training.

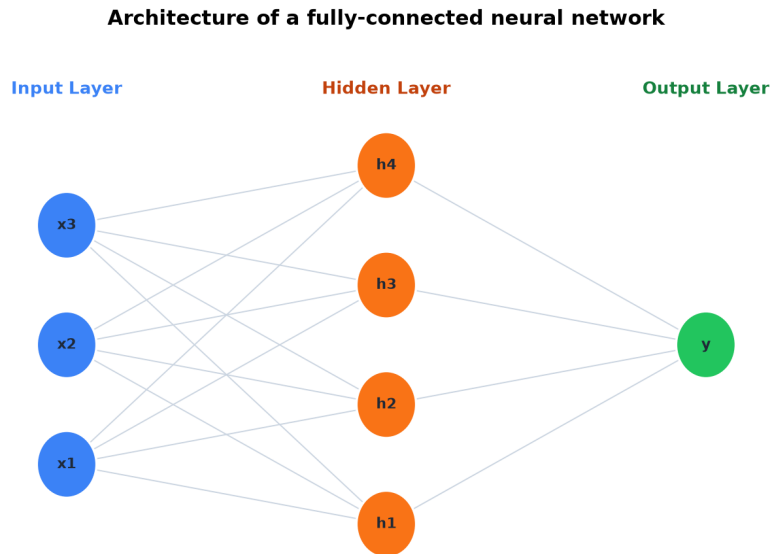


Figure2.13: Architecture of a fully-connected neural network.

2.2.6 Evaluating classification performance

Evaluation is the fundamental process for measuring a model's generalization ability, i.e., its aptitude for making accurate predictions on data it has never seen before. A model that performs exceptionally on training data but fails on new data is suffering from overfitting and lacks practical utility.

Validation protocols and data splitting

To measure generalization reliably, it is mandatory to split the available dataset into independent partitions.

- **Training, validation, and test:** in situations of data abundance, a training set is reserved to fit the model's weights, a validation set to select the best architecture or hyperparameters, and a test set that is kept in a "vault" for a final, unbiased evaluation.
- **K -fold Cross-Validation:** the recommended strategy when data is scarce. It splits the data into K partitions; the model is trained K times, each time using a different partition for validation and the remaining $K - 1$ for training. The final result is the average of the scores obtained across the K experiments, which reduces the variance of the estimate.
- **Stratification:** in classification tasks, it is vital to ensure that each partition preserves the original class proportion, especially when classes are imbalanced.

Success metrics for classifiers

The choice of metric must align with the business objective and the nature of the data.

The confusion matrix

A two-dimensional tool where rows represent the true classes and columns the predicted ones. It allows four essential values to be identified:

- **True Positives (TP)**: positive instances correctly classified.
- **True Negatives (TN)**: negative instances correctly classified.
- **False Positives (FP)**: Type I error or “false alarm.”
- **False Negatives (FN)**: Type II error or “miss.”

Derived metrics

- **Accuracy**: the fraction of correct predictions over the total. Can be misleading on skewed datasets; for example, if 90% of samples belong to one class, a model that always predicts that class will have 90% accuracy without having learned anything.
- **Precision**: the model’s ability to avoid labeling a negative sample as positive: $\frac{TP}{TP+FP}$
- **Recall**: the ability to find all the actual positive samples: $\frac{TP}{TP+FN}$
- **F1-score**: the harmonic mean of precision and recall, useful when seeking a balance between the two.
- **ROC AUC**: the area under the receiver operating characteristic curve measures the probability that the model ranks a random positive sample above a random negative one. A value of 1.0 is perfect and 0.5 is equivalent to random classification.

Fig. 2.14 compares a good classifier (curve far from the diagonal, high AUC) against the behavior of a random classifier (diagonal line, AUC = 0.5).

Training and diagnostic strategies

Evaluation does not only happen at the end — it should guide the entire development process.

- **Beating a baseline**: before starting with complex models, a trivial baseline (such as a random classifier) should be established. If the model cannot beat this common-sense threshold, it is likely that the data does not contain enough information, or the approach is flawed.
- **Monitoring learning curves**: plotting loss and accuracy for both training and validation makes it possible to detect the exact point of overfitting (when training loss decreases but validation loss starts to rise).
- **Early stopping**: a strategy that uses a callback to automatically halt training when the validation metric stops improving, saving time and preventing overfitting.
- **Error analysis**: manually inspecting the samples where the model fails helps understand which specific patterns the system is confusing (for example, confusing a “3” with a “5” in digit recognition).
- **A/B testing**: after deployment, randomized tests are recommended to measure the model’s real impact compared to the previous process.

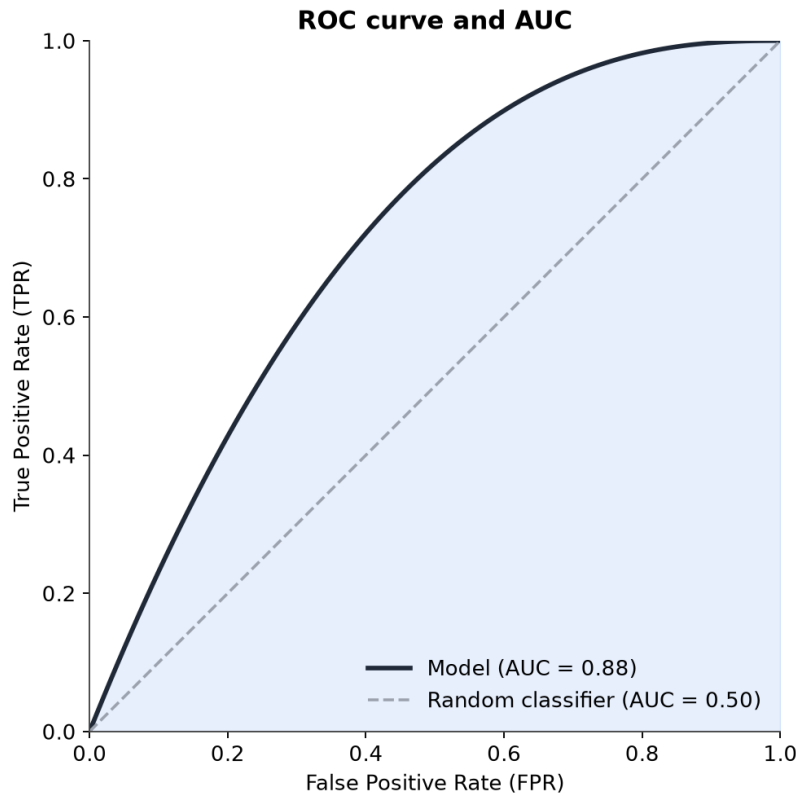


Figure 2.14: ROC curve and area under the curve (AUC).

2.2.7 Hands-on example in Java with SMILE

The `programacion-avanzada-smile` repository includes the full example `Ejemplo03Clasificacion.java`, which trains several classifiers on the *Iris* dataset with the `SMILE` library. The following fragment summarizes its main steps (comments translated into English).

SMILE offers two API styles: **formula-based** (`fit(Formula, DataFrame)`), used by tree-based models, which accept categorical variables without encoding) and **array-based** (`fit(double[][] x, int[] y)`), used by KNN, SVM, logistic regression, or neural networks).

```
import smile.classification.KNN;
import smile.classification.LogisticRegression;
import smile.classification.RandomForest;
import smile.data.DataFrame;
import smile.data.formula.Formula;
import smile.io.Read;
import smile.math.MathEx;
import smile.validation.CrossValidation;
import smile.validation.metric.Accuracy;
import smile.validation.metric.ConfusionMatrix;

MathEx.setSeed(42); // global seed: reproducible splits and models

DataFrame iris = Read.csv("data/iris.csv", "header=true").factorize("species");
Formula formula = Formula.lhs("species"); // species ~ all other columns

Utiles.Particion particion = Utiles.split(iris, 0.7);
```

(continues on next page)

(continued from previous page)

```

DataFrame train = particion.train();
DataFrame test = particion.test();

// 1. Random Forest (formula API): works directly on the DataFrame
RandomForest bosque = RandomForest.fit(formula, train, new RandomForest.Options(200));

int[] yTest = formula.y(test).toIntArray();
int[] prediccion = new int[test.nrow()];
for (int i = 0; i < test.nrow(); i++) {
    prediccion[i] = bosque.predict(test.get(i));
}
System.out.printf("Accuracy = %.2f%%\n", 100.0 * Accuracy.of(yTest, prediccion));
System.out.println(ConfusionMatrix.of(yTest, prediccion));

// 2. Array-based models: KNN and logistic regression
double[][] xTrain = formula.x(train).toArray();
int[] yTrain = formula.y(train).toIntArray();
double[][] xTest = formula.x(test).toArray();

KNN<double[]> knn = KNN.fit(xTrain, yTrain, 5);
LogisticRegression logistica = LogisticRegression.fit(xTrain, yTrain,
    new LogisticRegression.Options(0.1, 1E-5, 500)); // lambda, tolerance,
    iterations
System.out.printf("KNN (k=5)          accuracy = %.2f%%\n",
    100.0 * Accuracy.of(yTest, knn.predict(xTest)));
System.out.printf("Logistic regression accuracy = %.2f%%\n",
    100.0 * Accuracy.of(yTest, logistica.predict(xTest)));

// 3. 10-fold cross-validation: the honest estimate of performance
var cv = CrossValidation.classification(10, formula, iris,
    (f, datos) -> RandomForest.fit(f, datos, new RandomForest.Options(200)));
System.out.printf("Mean accuracy = %.4f (std. %.4f)\n",
    cv.avg().accuracy(), cv.std().accuracy());

```

A few things worth noticing when you run it:

- `factorize("species")` turns the text column with the species into a categorical variable with integer codes, which is what the classifiers expect.
- `ConfusionMatrix.of` builds the confusion matrix seen in this section, and `Accuracy.of` computes accuracy from the true and predicted labels.
- The first parameter of `LogisticRegression.Options` is λ , the strength of the L_2 regularization applied to the model's weights.
- The full example also shows soft prediction (posterior probabilities per class), saving the model to disk, and the use of encoders (`LEVEL`, `ONE_HOT`) for categorical variables on a second dataset.

To run it from the root of the repository: `mvn exec:java -Dexec.mainClass=es.usal.smile.Ejemplo03Clasificacion`.

2.2.8 Summary

- **Logistic Regression:** an extension of linear regression for classification.
 - **Confusion Matrix:** visualizes performance.
 - **Metrics:** choose based on what matters (precision or recall).
 - **Decision Trees:** interpretable but prone to overfitting.
 - **Random Forest:** an ensemble that reduces overfitting.
 - **SVM:** finds an optimal separation using the kernel trick.
 - **Ensembles:** combine several classification algorithms.
 - **Neural Networks:** powerful but complex.
-

Next: in Section 3 we will explore unsupervised learning, where we work with unlabeled data.

Part III

Section 3 - Unsupervised Learning

UNSUPERVISED LEARNING

Welcome to the third and final main section of the course. Here we will explore techniques for extracting patterns from **unlabeled data**.

What is unsupervised learning?

Unlike the previous blocks focused on supervised learning, where training is guided by a clear target variable or label, unsupervised learning represents the challenging task of dealing with datasets that lack any kind of expected output or annotation. In this paradigm, we only have a collection of input features X (or predictor variables), but we do not have a “teacher” or supervisor providing the correct answers to guide the learning process.

Technically, while supervised learning seeks to approximate a direct mapping function between inputs and outputs, or to estimate a **conditional distribution** $p(y|x)$, unsupervised learning works “blindly” to discover the **latent structure**, the intrinsic statistical associations, or the **unconditional distribution** of the data itself, $p(x)$. As computer scientist Yann LeCun famously illustrated: “If intelligence is a cake, unsupervised learning is the cake, supervised learning is the icing, and reinforcement learning is the cherry on top.” This analogy underscores that the vast majority of information available in the real world is unlabeled, making this paradigm the fundamental pillar on which true data understanding is built.

Main types of unsupervised learning

1. **Clustering**: aims to group similar data together.
2. **Dimensionality reduction**: simplifies complex data.
3. **Anomaly detection**: identifies rare or unusual cases.

Practical examples and uses

Unsupervised learning does not seek to make pointwise quantitative or qualitative predictions. Its value lies in extracting knowledge, simplifying structures, and revealing hidden patterns so they can be interpreted by humans or serve as preparation for other models:

- **Market segmentation**: by analyzing large volumes of demographic and transactional consumer data (such as income, occupation, or spending habits), companies can group people into homogeneous niches to target specific advertising campaigns or develop personalized products without needing to define these categories in advance.
- **Visualizing complex data**: projecting extremely high-dimensional datasets (with hundreds of attributes) into readable two- or three-dimensional representations, preserving the structure and geometric relationships so an analyst can intuitively understand how the data is organized.
- **Recommendation systems and collaborative filtering**: grouping users with similar tastes or products with similar characteristics to predict implicit preferences based on historical consumption behavior.

- **Anomaly detection:** modeling the statistical behavior of “normal” data to automatically identify suspicious instances that deviate significantly from the general pattern (for example, to detect financial fraud or manufacturing line failures).

What Will You Find in This Section?

- **Clustering:**
 - K -means: centroid-based grouping, initialization (K -means++), and choosing K with the elbow method.
 - Hierarchical clustering: the agglomerative approach, dendrograms, and linkage criteria.
 - DBSCAN: density-based clustering, able to detect arbitrary shapes and noise.
 - Evaluation techniques: silhouette coefficient, internal metrics, information-theoretic criteria, and external evaluation with labeled data.
- **Dimensionality reduction:**
 - The curse of dimensionality: why working in spaces with many variables is a problem.
 - PCA: linear projection that maximizes retained variance, SVD, and reconstruction.
 - Manifold learning: the manifold hypothesis and the Swiss roll example.
 - Non-linear methods: Kernel PCA, t-SNE, and UMAP.
- **Anomaly detection:**
 - Anomaly detection vs. novelty detection: the difference based on training-data contamination.
 - Statistical and reconstruction-based approaches: Gaussian mixture models (GMM) and PCA reconstruction error.
 - Isolation forest: isolating observations through random partitions.
 - Local Outlier Factor (LOF): detecting local anomalies based on neighborhood density.
 - One-class SVM: delimiting the region of normal behavior.

Important Note

Unsupervised learning is more **art than science**. There is no single “correct” answer. Your domain interpretation and external feedback are crucial.

Let’s start: go to Chapter 3.1 to learn about clustering, the most popular unsupervised approach.

3.1 Clustering

3.1.1 Introduction

Unsupervised learning represents a fundamental paradigm of artificial intelligence, characterized by the absence of labels or target variables to guide the training process. In this block, the system does not receive a “ground-truth” criterion; instead, it must explore the intrinsic mathematical relationships within the data to reveal its hidden structure.

Within this paradigm, **clustering** (or grouping) is the most widespread technique, aimed at partitioning a dataset into subgroups whose members share high internal similarity while, at the same time, being highly differentiated from members of other subgroups.

3.1.2 Clustering algorithms

Several **clustering** algorithms exist. The most relevant ones today are explained below.

The K -means algorithm

K -means [Mac67] is a grouping algorithm that performs partitions based on centroids. The algorithm is an iterative grouping technique that seeks to partition a set of observations into K distinct clusters. It is a geometric approach based on the concept of centroids, which act as the center of gravity of each group.

Fig. 3.1 summarizes the algorithm's cycle: after assigning each sample to its centroid and computing the inertia (WCSS), the centroids are updated and the process repeats until convergence.

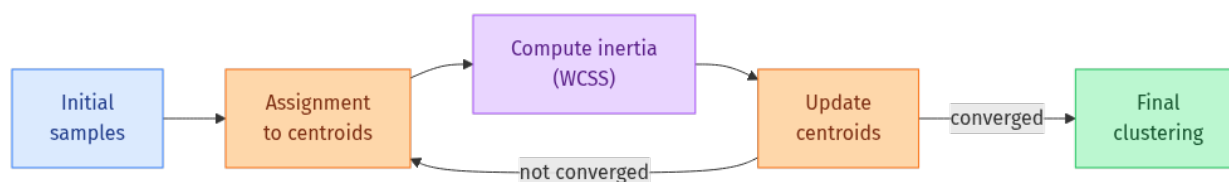


Figure 3.1: Iterative cycle of K -means.

Algorithmic mechanics and convergence

The mathematical goal of K -means is to minimize intra-cluster inertia, formally known as the Within-Cluster Sum of Squares (WCSS). The optimization process is divided into the following iterative steps:

1. **Initialization:** K points are defined in the feature space to act as initial centroids.
2. **Assignment step** (expectation): each sample in the dataset is assigned to the nearest centroid using a distance metric, typically the Euclidean distance: $d(x, y) = \|x - y\|_2 = \sqrt{\sum_{i=1}^D (x_i - y_i)^2}$
3. **Update step** (maximization): the position of each centroid is recalculated as the mathematical average (the arithmetic mean) of all samples assigned to that cluster.
4. **Convergence:** steps 2 and 3 are repeated iteratively until the centroids no longer change position significantly, or the sample assignments remain stable.

Fig. 3.2 shows the three stages on a dataset with three natural groups: the centroids (marked with an X) start from random positions and converge toward the center of each group after just a few iterations.

The local minima and initialization problem (K -means++)

K -means is a heuristic algorithm and is therefore highly sensitive to the random initialization of its centroids. If the initial centroids are placed in unfavorable regions of the feature space, the algorithm can converge to suboptimal local minima.

To mitigate this vulnerability, K -means++ initialization is used:

- The first centroid is selected uniformly at random among the data points.
- Subsequent centroids are chosen probabilistically, where the probability of selecting a point as the new centroid is proportional to the squared distance to the nearest existing centroid: $P(x) \propto D(x)^2$

This smart-seeding method ensures the initial centroids are widely spread across the feature space, speeding up convergence and improving the quality of the final grouping.

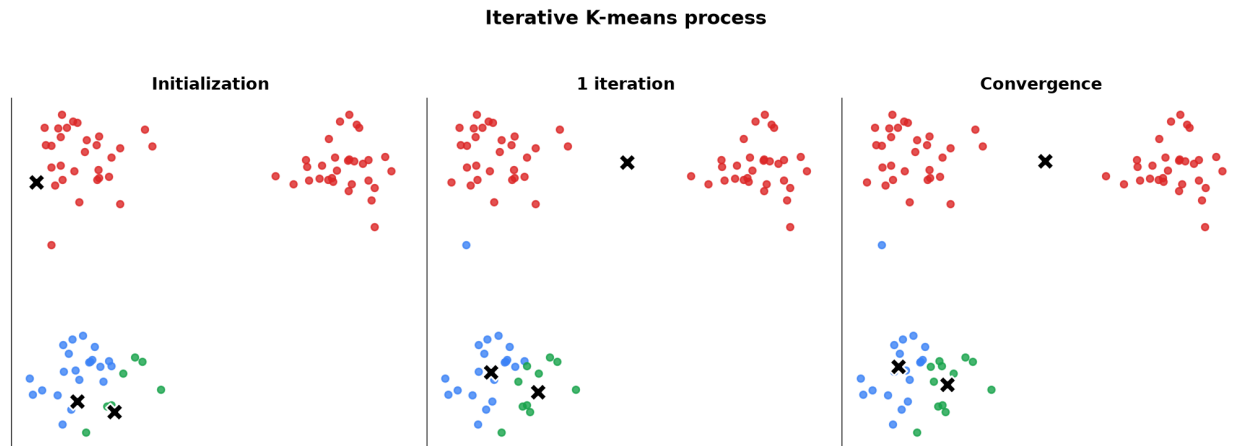


Figure 3.2: Iterative K-means process: from initialization to convergence.

Selecting model complexity

The elbow method

Since the number of clusters K is a hyperparameter that must be defined by the user before training, a scientific protocol is required to select it.

The elbow method consists of plotting the WCSS value as a function of different values of K :

- As K increases, WCSS naturally decreases because the clusters become smaller and points move closer to their respective centroids.
- The optimal value of K is located at the inflection point of the curve (the “elbow”), where the decrease in inertia stops being abrupt and becomes linear. This represents an optimal balance between model complexity and group cohesion.

Fig. 3.3 shows a typical example: inertia drops sharply up to $K = 3$ and barely improves afterward, signaling that 3 is the most reasonable number of clusters for this data.

Hierarchical clustering: agglomerative and dendrograms

Hierarchical **clustering** groups data without needing to define the number of groups K in advance. Its main advantage is that it produces a hierarchical grouping structure that can be easily visualized.

Agglomerative approach (bottom-up)

The most common method is agglomerative hierarchical clustering, which operates bottom-up. The algorithm starts by treating each individual observation as its own independent cluster. At each iterative step, the distances between all clusters are computed and the two closest groups are merged. This merging process repeats successively until all samples are integrated into a single global cluster.

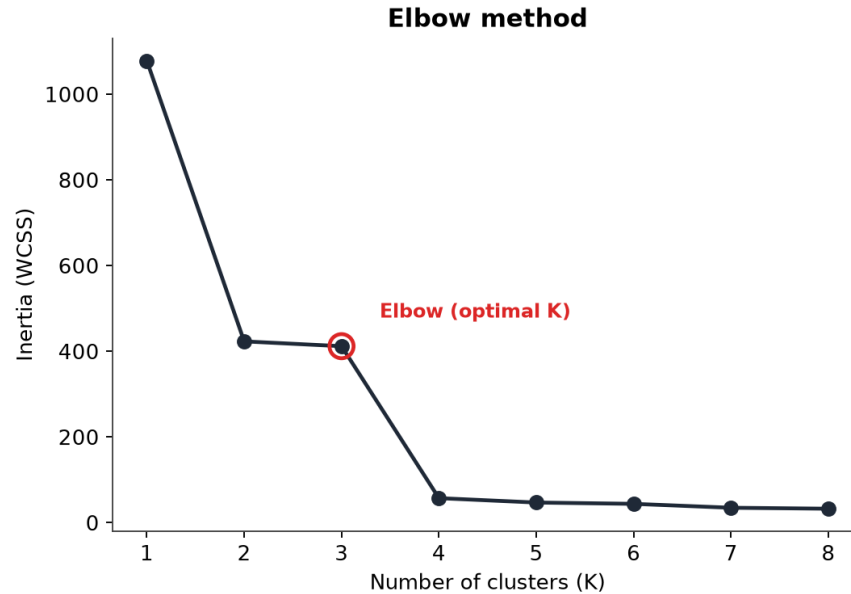


Figure3.3: Elbow method: inertia stops decreasing sharply beyond K=3.

Dendrograms and interpreting cuts

The hierarchy of merges is graphically represented by a tree diagram called a dendrogram. The vertical axis of the dendrogram represents the merge distance (or cophenetic dissimilarity) between subgroups.

The horizontal axis arranges individual observations so similar branches are adjacent. The user can determine the final number of clusters by making a horizontal cut across the dendrogram at a specific dissimilarity height. The number of vertical lines intersected by the cut defines the resulting number of groups, offering an interpretive flexibility that rigid algorithms like K -means lack.

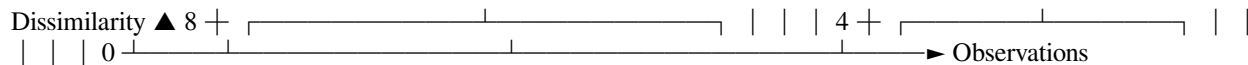


Fig. 3.4 shows a real dendrogram built on data with three groups: the dashed red line marks a cut that produces exactly 3 clusters (one per color).

Linkage criteria

To determine which clusters should be merged at each step, we must define how to measure distance or dissimilarity between groups containing multiple observations. The main linkage methods are:

- **Complete linkage:** computes the maximum distance between any member of the first cluster and any member of the second cluster: $D(A, B) = \max_{x \in A, y \in B} d(x, y)$ It produces compact dendrograms and well-balanced, spherically shaped clusters.
- **Single linkage:** computes the minimum distance between any member of the first cluster and any member of the second cluster: $D(A, B) = \min_{x \in A, y \in B} d(x, y)$ It is sensitive to noise and can produce a chaining effect, where clusters merge into elongated, diffuse shapes.
- **Average linkage:** computes the average of all distances between each point in the first cluster and each point in the second: $D(A, B) = \frac{1}{|A||B|} \sum_{x \in A} \sum_{y \in B} d(x, y)$ It is a robust criterion that balances the stability of complete linkage with the tolerance of single linkage.

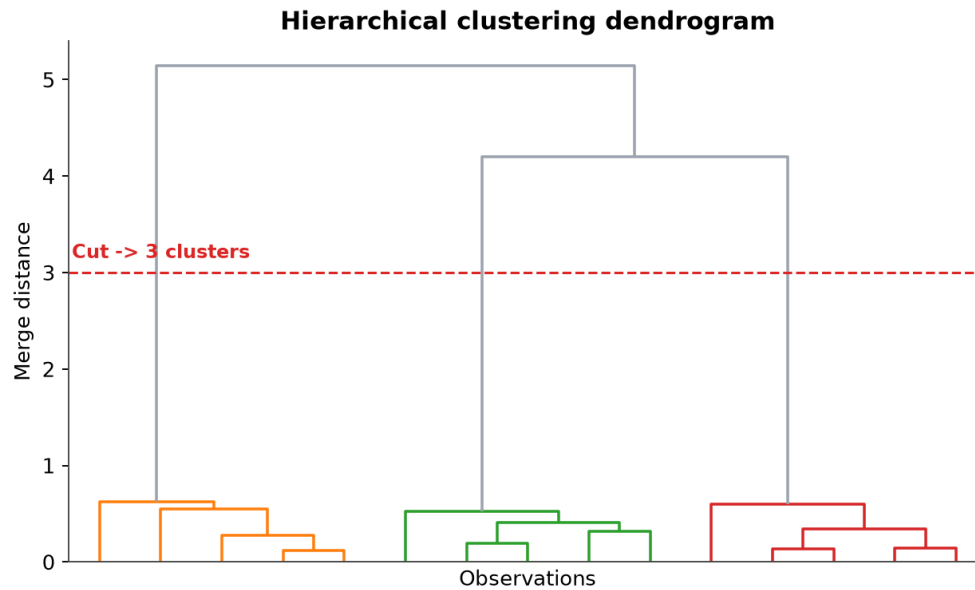


Figure 3.4: Hierarchical clustering dendrogram with a cut that produces 3 clusters.

Fig. 3.5 compares the three linkage criteria on real gene-expression data from the *NCI60* dataset (cancer cell lines): note how single linkage (bottom) produces the characteristic chaining effect, while complete and average linkage (top and middle) yield more balanced groupings.

DBSCAN: density-based clustering

The DBSCAN algorithm (Density-Based Spatial Clustering of Applications with Noise) [EKSX96] offers a radically different approach from *K*-means and hierarchical **clustering** by defining groups based on the local density of the data in the feature space. This allows it to discover clusters of arbitrary geometric shapes and naturally isolate noisy samples.

Foundations and critical parameters

DBSCAN requires setting two essential hyperparameters that define the local density of the neighborhood:

- **ϵ (Epsilon):** the neighborhood radius defining the surroundings around any point.
- **MinPts** (minimum samples): the minimum number of points that must exist within radius ϵ for that region to be considered dense.

Classifying points in DBSCAN

During execution, DBSCAN examines every point in the dataset and classifies it into one of three exclusive categories based on the density conditions of its neighborhood:

- **Core points:** a point is labeled a core point if its ϵ -radius neighborhood contains a number of samples equal to or greater than MinPts.
- **Border points:** points that do not meet the minimum density requirement of MinPts to be considered core, but reside within the ϵ -neighborhood of a point that is a core point.
- **Noise points** (outliers): any point that is classified as neither core nor border. These points are considered anomalies or outliers and are not assigned to any cluster.

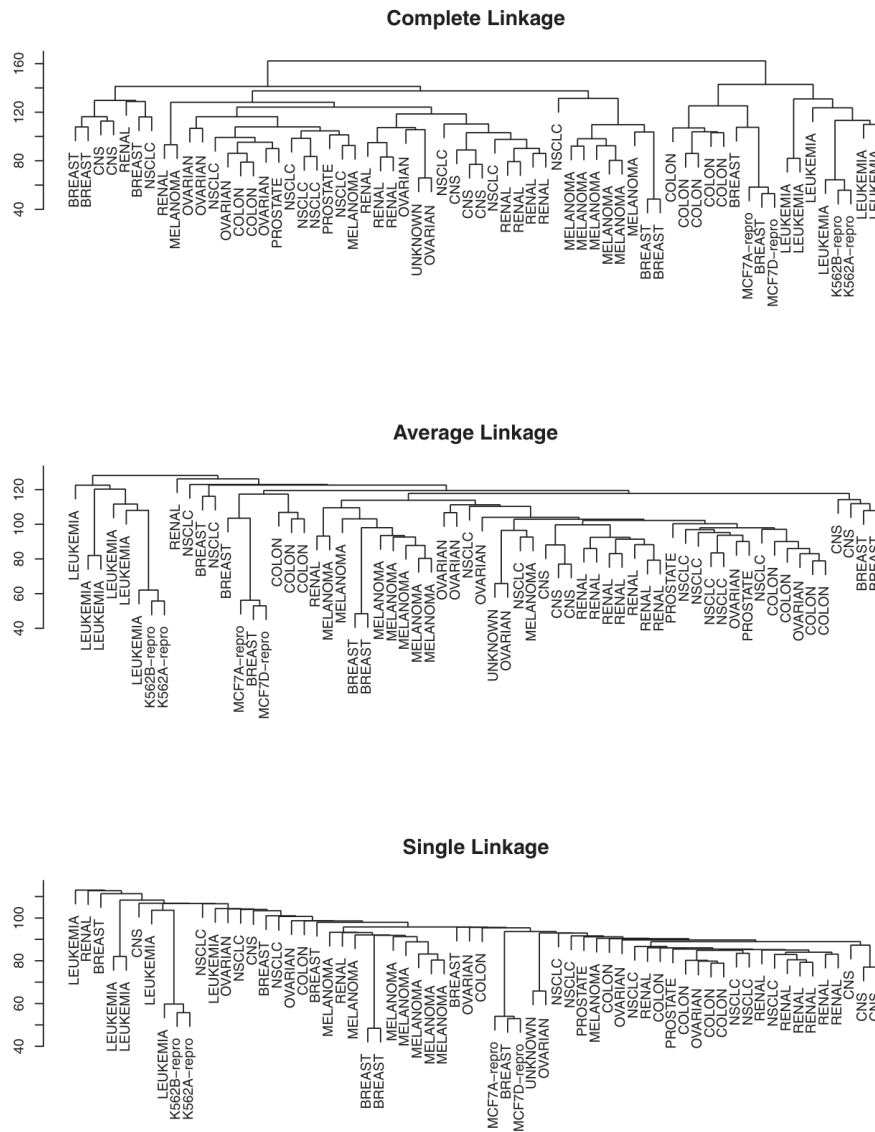


Figure3.5: Hierarchical clustering of the NCI60 dataset with complete, average, and single linkage. Source: James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*, Figure 10.17. Springer. Freely distributed for educational use (statlearning.com).

Fig. 3.6 illustrates the three types with $\epsilon = 1$ and $\text{MinPts} = 4$: the blue points have at least 4 neighbors within their circle (core); the orange ones fall inside the circle of a core point but do not reach MinPts on their own (border); and the red cross has no core point nearby (noise).

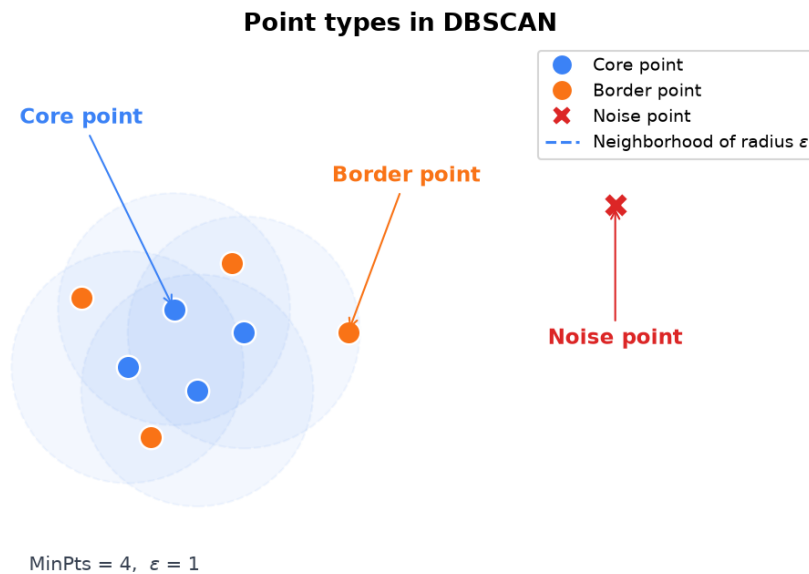


Figure3.6: Classification of points in DBSCAN: core, border, and noise.

Advantages over distance-based methods

- **Robustness to noise:** unlike K -means, which forces every sample to belong to a cluster (artificially shifting centroids in the presence of outliers), DBSCAN natively identifies and isolates noise.
- **Geometric flexibility:** it does not assume that clusters must be spherical; it can discover nested, elongated, or complex-shaped low-dimensional clusters within high-dimensional spaces.

Fig. 3.7 compares both algorithms on non-convex data: K -means arbitrarily cuts the group in half, while DBSCAN respects the true shape defined by density.

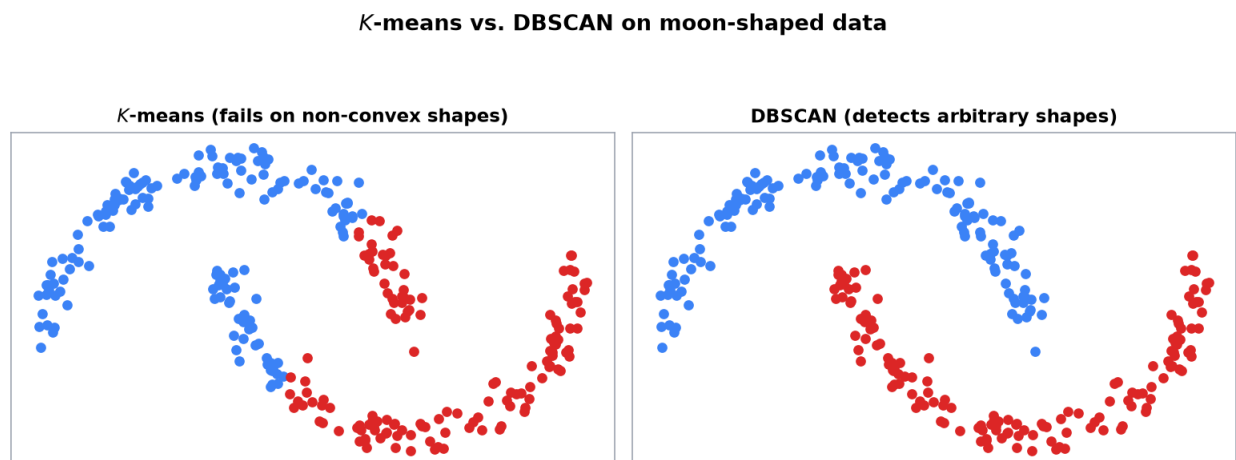


Figure3.7: K-means vs. DBSCAN on non-convex shaped data.

3.1.3 Evaluation techniques in clustering

Evaluating grouping structures is one of the most complex and debated areas of unsupervised learning, since, lacking class labels or a “ground-truth” during training, there is no single, objective success criterion. The quality of a grouping is often measured subjectively based on the model’s usefulness to the end user in their application domain.

However, to approximate this analysis rigorously, the scientific literature has consolidated evaluation methods into four broad approaches: (i) internal geometric metrics, (ii) information-theory-based criteria, (iii) unified information metrics and approaches, and (iv) supervised external validations.

Internal evaluation metrics and geometric structure

These analyze the spatial arrangement of points in the feature space to measure two desirable properties: that the clusters are as compact as possible (low internal variance) and that they are well separated from each other.

Silhouette coefficient and score

The silhouette coefficient (sc) [Rou87] evaluates the quality of each sample’s assignment individually, serving as a diagnostic for predominantly spherical groupings. For an instance i , it is computed as:

$$sc(i) = \frac{b_i - a_i}{\max(a_i, b_i)}$$

Where:

- a_i (**cohesion**): the average (usually Euclidean) distance between instance i and all other points belonging to its own cluster. We want this value to be as low as possible.
- b_i (**separation**): the average distance between instance i and all samples in the nearest neighboring cluster (the cluster that minimizes this average distance, excluding its own). We want this value to be as high as possible.

Interpreting the results

- The coefficient strictly ranges over $[-1, 1]$.
- A value close to $+1$ indicates the sample is well inside its own group and far from the boundaries of other clusters.
- A value close to 0 indicates the sample sits on the decision boundary between two clusters.
- A value close to -1 strongly suggests the sample has been assigned to the wrong group.

The overall silhouette score is the average of the coefficients across all instances in the dataset. In a silhouette diagram, coefficients are sorted from highest to lowest and plotted per cluster. The height of each silhouette (knife shape) indicates the cluster size, and the width represents the individual coefficient of its points. This allows for visually identifying whether any cluster is excessively large or has too many samples below the overall average score (represented by a dashed vertical line), which reveals low-quality groupings.

Fig. 3.8 shows a real silhouette diagram with three clusters: most samples in cluster 1 (red) and 0 (blue) exceed the average silhouette, while cluster 2 (green) is more heterogeneous.

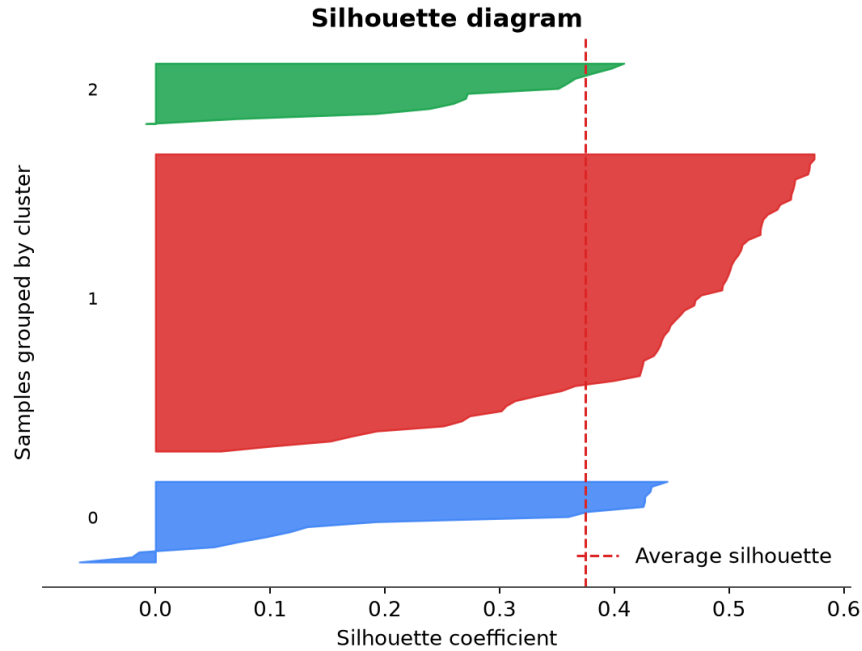


Figure 3.8: Silhouette diagram: each “knife” represents a cluster and its grouping quality.

Gap statistic

Proposed by Tibshirani et al. [TWH01], the **gap statistic** mathematically formalizes the search for the optimal number of clusters (K), overcoming the visual vagueness of the heuristic elbow method.

It compares the log of the observed inertia curve of your data ($\log W_K$) with the mathematical expectation of the inertia computed over multiple artificially generated samples drawn from a uniform distribution (with no grouping structure, i.e., the null hypothesis) over the hyper-rectangle enclosing the real data:

$$\text{Gap}(K) = E^*[\log W_K] - \log W_K$$

The method estimates the optimal K by maximizing this “gap.” Mathematically, the formal rule selects the smallest K such that:

$$K^* = \operatorname{argmin}_K \{K \mid \text{Gap}(K) \geq \text{Gap}(K+1) - s'_{K+1}\}$$

Where $s'_{K+1} = s_{K+1} \sqrt{1 + 1/B}$ represents the standard deviation of the simulation adjusted for the number of artificial replicates B . Its greatest advantage is that it can determine whether the optimal number of clusters is 1 (i.e., that the data has no natural clusters), a scenario where inertia or the silhouette coefficient fail.

Cophenetic correlation coefficient

This technique evaluates the quality of the hierarchical structure imposed by hierarchical clustering. It measures the Pearson correlation between the initial dissimilarity matrix $\{d_{ii'}\}$ of the observations and their corresponding cophenetic dissimilarities $\{C_{ii'}\}$ obtained from the merge heights in the dendrogram.

The cophenetic distance $C_{ii'}$ is the exact height of the link in the dendrogram where observations i and i' are first joined into a common cluster. Since cophenetic distances must strictly satisfy the ultrametric inequality ($C_{ii'} \leq \max\{C_{ik}, C_{i'k}\}$), it is uncommon for real input data to fit this metric constraint perfectly. A low cophenetic correlation warns the designer that the dendrogram is forcing an artificial hierarchy where the real data does not exhibit one.

Probabilistic and information-theory criteria

When working with probability-based grouping models (such as Gaussian Mixture Models, or GMM, trained via the EM algorithm), pure geometric distances become unreliable because groups may adopt elliptical shapes, oblique orientations, or highly variable densities.

Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC)

If we evaluate a probabilistic model solely by its log-likelihood (LL), the model with the largest number of parameters or components will always appear better on the training data, promoting overfitting. To prevent this, AIC and BIC add mathematical penalties that balance fit and complexity:

$$AIC = -2LL + 2d$$

$$BIC = -2LL + d \log(N)$$

Where LL is the model's maximum log-likelihood, d is the total number of free parameters to learn (means, weights, and covariances of the components), and N is the sample size.

Penalization

Since $\log N > 2$ for any real dataset ($N > 7$), BIC penalizes complexity much more severely than AIC, favoring more parsimonious models (with fewer components).

Asymptotic behavior

BIC is asymptotically consistent, meaning that if the true model is among the candidates, the probability of selecting it tends to 1 as $N \rightarrow \infty$. AIC, in contrast, tends to overfit on infinite samples, choosing overly complex models, although it behaves more competitively than the conservative BIC on small samples.

WAIC (Widely Applicable Information Criterion)

AIC and BIC assume that the model's parameters approximate a Gaussian distribution and that the model is non-singular. However, in probabilistic mixtures and deep learning models with correlated or redundant parameters (singular models), these assumptions break down.

WAIC is a Bayesian information metric that estimates the expected log pointwise predictive density (ELPD) via Monte Carlo simulation methods, penalizing complexity based on the posterior variance of each data point's predictions. This provides a robust evaluation framework that works even under singular and over-parameterized geometries.

Unified information metrics and approaches

The minimum description length (MDL) principle

Inspired by information theory and data coding, the MDL principle evaluates clustering under the premise of data compression. If a grouping is natural and of high quality, it should let us describe or transmit the dataset more compactly (using fewer bits of information) than if we described the points without grouping them.

To transmit a set of points using MDL, we must encode two elements:

- The theory or model: the spatial location of the cluster centroids and their respective assignments in bits ($\log_2 K$ bits per sample).

- The data given the model: the feature values of each instance, expressed only as the residual deviation from its assigned centroid.

If the segmentation captures dense, genuine groups, the drastic reduction in bits needed to describe the small individual residuals will more than compensate for the extra cost of transmitting the centroid positions, achieving a minimal total description. If the grouping is not genuine, the total bit cost will increase, signaling that the subdivision is useless.

Category utility

Used in incremental and hierarchical clustering methods such as Cobweb or Classit, category utility (CU) measures the increase in the ability to correctly predict a sample's attributes after learning it belongs to a specific cluster, compared to the unconditional prediction of the data.

For categorical or nominal attributes, it is formulated as:

$$CU(C_1, \dots, C_k) = \frac{\sum_{l=1}^k \Pr[C_l] \sum_i \sum_j (\Pr[a_i = v_{ij} | C_l]^2 - \Pr[a_i = v_{ij}]^2)}{k}$$

Where $\Pr[a_i = v_{ij} | C_l]$ is the conditional probability that attribute i takes the value v_{ij} in cluster l , and $\Pr[a_i = v_{ij}]$ is its overall unconditional probability.

Avoiding fragmentation (dividing by k)

Without the denominator k , the maximum score would be obtained by assigning each individual instance to its own exclusive cluster, where the conditional probability is 1.0. Dividing by k acts as a heuristic factor that penalizes excessive fragmentation of the model.

Minimum variance (acuity)

For continuous attributes, the formula assumes a normal distribution and requires estimating local and global standard deviations. To prevent a cluster with a single element (or zero variance) from producing an infinite, undefined value in the equation, a minimum variance parameter, or acuity, is imposed, emulating sensor measurement error.

External evaluation using labeled data

When external reference labels are available (labels that were not used to guide the clustering), we can measure how well the discovered boundaries align with known biological or business classes.

Purity

Consists of assigning each entire cluster the most frequent class label within it and computing the proportion of correctly classified elements. It is formally defined as:

$$\text{Purity} = \sum_i \frac{N_i}{N} \max_j (p_{ij})$$

Where N_i is the size of cluster i , N is the total data count, and $p_{ij} = N_{ij}/N_i$ is the fraction of cluster i occupied by class j .

Limitation

Purity does not penalize the number of clusters. If you assign each sample individually to its own cluster ($K = N$), purity will trivially equal 1.0, invalidating its use unless the number of clusters is independently controlled.

Rand Index (RI) and Adjusted Rand Index (ARI)

The Rand index analyzes grouping consistency based on pairs of elements. It examines every possible pair of observations in the dataset and evaluates whether the decision to assign them to the same cluster or to different clusters was consistent between the estimated partition (U) and the reference partition (V):

$$R = \frac{TP+TN}{TP+FP+FN+TN}$$

- **TP** (True Positives): number of pairs that are in the same cluster in U and in the same class in V .
- **TN** (True Negatives): number of pairs that are in separate clusters in U and in different classes in V .
- **FP** (False Positives): number of pairs assigned to the same cluster in U but to different classes in V .
- **FN** (False Negatives): number of pairs in separate clusters in U but in the same class in V .

RI ranges between 0 (total disagreement) and 1 (perfect agreement). However, classic RI rarely reaches 0, since chance tends to inflate its score. The Adjusted Rand Index (ARI) corrects this by subtracting the expected agreement of two random classifications under a generalized hypergeometric distribution model, ensuring a random grouping scores 0.0 on average.

Classes-to-Clusters Evaluation

Popularized by classic software suites such as Weka, this approach trains the clustering model while completely ignoring the class attribute. Once the space has been segmented, the evaluator examines the composition of each cluster to retrospectively assign it the majority class label.

From this assignment, it is possible to map the geometric boundaries as if they were a supervised classifier and build a classic confusion matrix that reveals the exact rates of false positives, false negatives, and overall per-class accuracy.

3.1.4 Hands-on example in Java with SMILE

The `programacion-avanzada-smile` repository includes the full example `Ejemplo05Clustering.java`, which applies the three algorithms from this section with the `SMILE` library (comments translated into English). It uses the *Iris* dataset **with the species column removed**: the algorithm never sees the labels, but since we know there are three species we can compare the resulting partition against the true one using an external index (ARI).

```
import java.util.Arrays;
import smile.clustering.CentroidClustering;
import smile.clustering.Clustering;
import smile.clustering.DBSCAN;
import smile.clustering.HierarchicalClustering;
import smile.clustering.KMeans;
import smile.clustering.linkage.WardLinkage;
import smile.data.DataFrame;
import smile.io.Read;
import smile.math.MathEx;
import smile.validation.metric.AdjustedRandIndex;

MathEx.setSeed(42);
```

(continues on next page)

(continued from previous page)

```

DataFrame iris = Read.csv("data/iris.csv", "header=true").factorize("species");
double[][] x = iris.drop("species").toArray();           // no labels
int[] especieReal = iris.column("species").toIntArray(); // only for evaluation

// 1. K-means with k = 3: fit(data, k, maxIterations)
CentroidClustering<double[], double[]> kmeans = KMeans.fit(x, 3, 100);
int[] grupos = kmeans.group();
System.out.printf("Distortion (inertia) = %.3f\n", kmeans.distortion());
System.out.printf("ARI against the true species = %.4f\n",
    AdjustedRandIndex.of(especieReal, grupos));

// Assign a new sample to the nearest centroid
double[] nueva = { 5.9, 3.0, 5.1, 1.8 };
System.out.printf("The sample falls in cluster %d\n", kmeans.predict(nueva));

// 2. Elbow method: distortion for different values of k
for (int k = 2; k <= 8; k++) {
    System.out.printf("k = %d  distortion = %.3f\n", k, KMeans.fit(x, k, 100).
        distortion());
}

// 3. DBSCAN: fit(data, minPts, epsilon). No need to fix k, and it detects noise
DBSCAN<double[]> dbscan = DBSCAN.fit(x, 5, 0.8);
long ruido = Arrays.stream(dbscan.group()).filter(g -> g == Clustering.OUTLIER).
    count();
System.out.printf("DBSCAN: %d clusters, %d noise points\n", dbscan.k(), ruido);

// 4. Agglomerative hierarchical clustering (Ward linkage), cut into 3 groups
HierarchicalClustering jerarquico = HierarchicalClustering.fit(WardLinkage.of(x));
int[] particion3 = jerarquico.partition(3);
System.out.printf("Hierarchical: ARI = %.4f\n", AdjustedRandIndex.of(especieReal,
    particion3));

```

A few things worth noticing when you run it:

- `kmeans.distortion()` is the within-cluster inertia (WCSS) that K -means minimizes; the loop over k produces the values that would be plotted in the elbow-method chart.
- Points that DBSCAN considers noise receive the special label `Clustering.OUTLIER` instead of a cluster number.
- In hierarchical clustering, the linkage criterion is built first (`WardLinkage`, though `SingleLinkage`, `CompleteLinkage`, or `UPGMALinkage` for average linkage are also available) and the dendrogram is then cut with `partition(k)`.
- The full example also compares X-means (automatic choice of k via BIC) and shows the effect of standardizing the variables before clustering.

To run it from the root of the repository: `mvn exec:java -Dexec.mainClass=es.usal.smile.Ejemplo05Clustering`.

3.1.5 Summary

- ***K*-means**: simple, fast, requires a known K .
- **Elbow Method**: a heuristic for choosing K .
- **Hierarchical clustering**: dendrogram, better visualization.
- **DBSCAN**: detects arbitrary shapes and outliers.
- **Silhouette**: a metric for evaluating quality without labels.

Next: In 3.2 we will learn to reduce dimensionality to simplify and visualize complex data.

3.2 Dimensionality Reduction

3.2.1 Introduction

In modern data analytics and the design of artificial intelligence systems, it is common to face datasets characterized by hundreds or thousands of predictor variables. However, working directly in these multidimensional spaces presents serious practical and theoretical drawbacks. Dimensionality reduction systematically addresses this problem by seeking to project or compress the data into a low-dimensional space (usually 2D or 3D). This process not only drastically speeds up computation and mitigates overfitting, but is also an indispensable tool for **data visualization** (DataViz) and for extracting explainable latent factors.

3.2.2 The curse of dimensionality

Human intuition is wired to reason in three physical dimensions, which makes it difficult for us to understand the geometric properties of high-dimensional spaces. Many mathematical properties of traditional machine learning algorithms degrade or collapse due to this phenomenon, formally coined by Richard Bellman as the curse of dimensionality.

Fig. 3.9 shows the progression from a point to a p -dimensional hypercube: as the dimension grows, the space becomes ultra-sparse and samples tend to sit at the boundary.

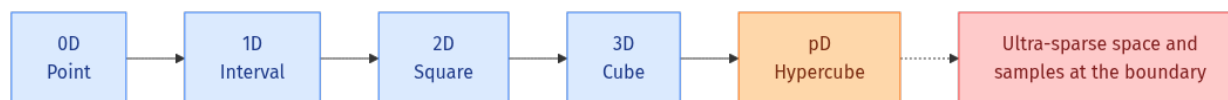


Figure 3.9: Progression of dimensions: from a point to a hypercube.

Loss of neighborhood and sparse space

The main symptom of the curse of dimensionality is that, as the dimension p increases, the volume of the space grows exponentially with respect to the features. This causes any real dataset, however large, to populate the input space extremely sparsely. To illustrate this fact:

- If in a single dimension ($p = 1$) a local range of 10% represents a representative neighborhood sample, to capture the same equivalent 10% volume in a 10-dimensional space ($p = 10$), a local algorithm (such as K -NN) must extend its search to cover 80% of the range of each variable.
- As a result, the points in the neighborhood stop being “local,” and the algorithm loses its statistical estimation power.

- To maintain a constant sampling density as predictor variables are added, the required dataset size grows exponentially ($O(N^p)$). In practice, obtaining such a volume of data is infeasible.

The geometric anomaly of the extremes

In high-dimensional spaces, geometry becomes highly counterintuitive.

- **Attraction to the boundary:** in a two-dimensional unit square (1x1), the probability that a randomly chosen sample lies near the edge (within 0.001 distance) is barely 0.4%. However, in a 10,000-dimensional hypercube, this probability exceeds 99.9999%. Practically all samples naturally reside at the edges and outer corners of the hypercube.
- **Uniformity of distances:** the average distance between two randomly chosen points in a hypercube grows drastically with dimensionality. Relative distances become equalized, making all samples appear to be almost the same distance from each other and neutralizing the effectiveness of standard metrics such as Euclidean distance. This severely increases the risk of overfitting in classifiers, since the model becomes unstable under slight variations.

Fig. 3.10 quantifies both effects through simulation: on the left, the percentage of samples near a hypercube's boundary grows rapidly with dimensions; on the right, the ratio between the minimum and maximum distance among random points tends to 1 — that is, all points appear equidistant.

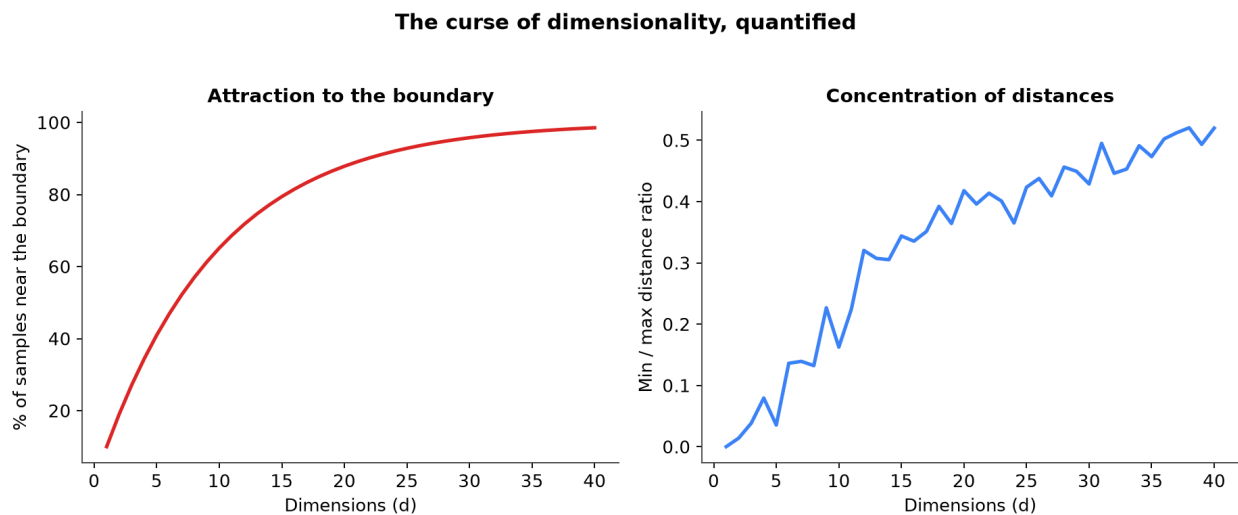


Figure 3.10: The curse of dimensionality, quantified through simulation.

3.2.3 Linear projection methods: PCA (Principal Components Analysis)

Principal Component Analysis (PCA), originally developed in the early 20th century by Pearson and Hotelling [Hot33, Pea01], is the quintessential linear dimensionality reduction algorithm. Its goal is to orthogonally project the original data $X \in \mathbb{R}^D$ onto a low-dimensional linear subspace $Z \in \mathbb{R}^M$ (where $M < D$), minimizing information loss under strict statistical criteria.

Maximum variance perspective

From a geometric point of view, the first principal component (PC1) is defined as the direction or axis of the feature space along which the data varies the most. By projecting the data onto this axis, the largest percentage of the original information's spread is preserved.

Fig. 3.11 compares two projections of the same data: onto PC1 (left) the projected points stay widely spread along the axis and 90% of the variance is preserved; onto another direction (right) the points pile up and only 13% is retained.

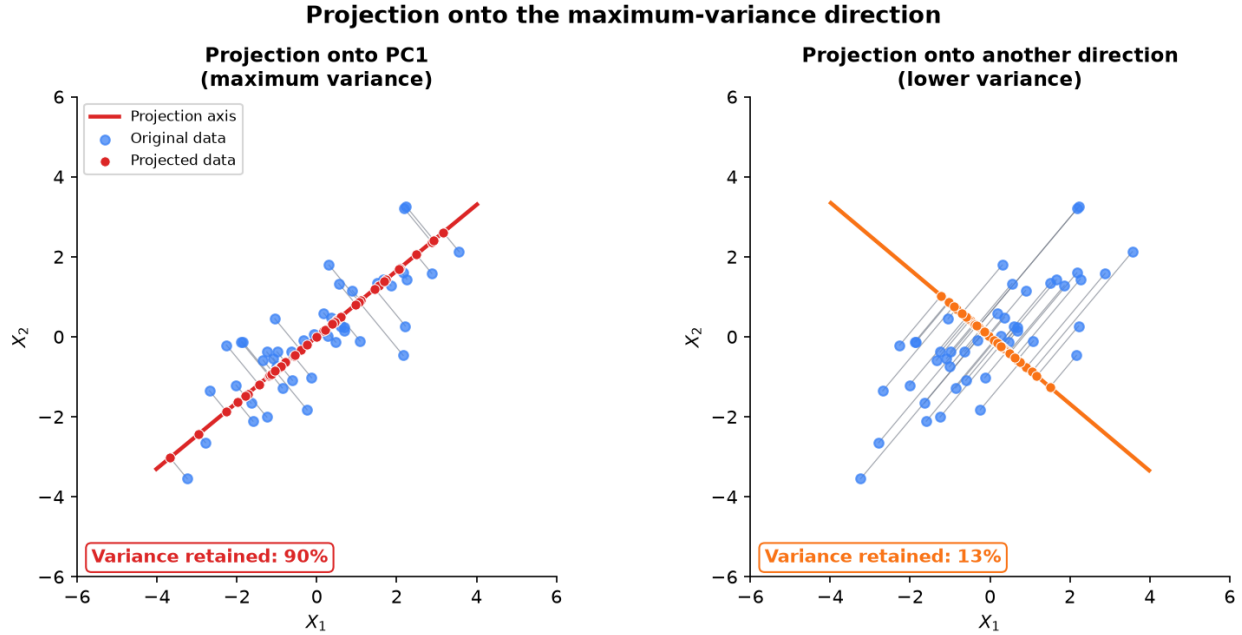


Figure3.11: Projection onto the maximum-variance direction (PC1) versus another direction.

The second principal component (PC2) seeks the direction that explains as much of the remaining variance as possible, under the strict constraint of being fully orthogonal to (and therefore uncorrelated with) the first component.

This procedure is repeated successively to generate up to D distinct components.

Fig. 3.12 shows this concept on real correlated data: the red arrow (PC1) points in the direction of maximum variance and the green one (PC2), orthogonal to it, captures the remaining variance.

Mathematical derivation and the SVD

To perform PCA, the original data must first be centered (subtracting the arithmetic mean of each variable) and, usually, standardized to have unit variance (preventing variables with arbitrarily large metric scales from dominating the optimization).

The covariance matrix of the data is defined as:

$$S = \frac{1}{N} X X^T$$

Through eigenvalue decomposition of the covariance matrix (or via Singular Value Decomposition — SVD — of the original data matrix X), the principal directions are extracted:

$$S = V D^2 V^T$$

Where the columns of matrix V correspond to the loading vectors, representing the directions of the principal components. The eigenvalue λ_m corresponding to each eigenvector directly measures the amount of variance explained by that specific component.

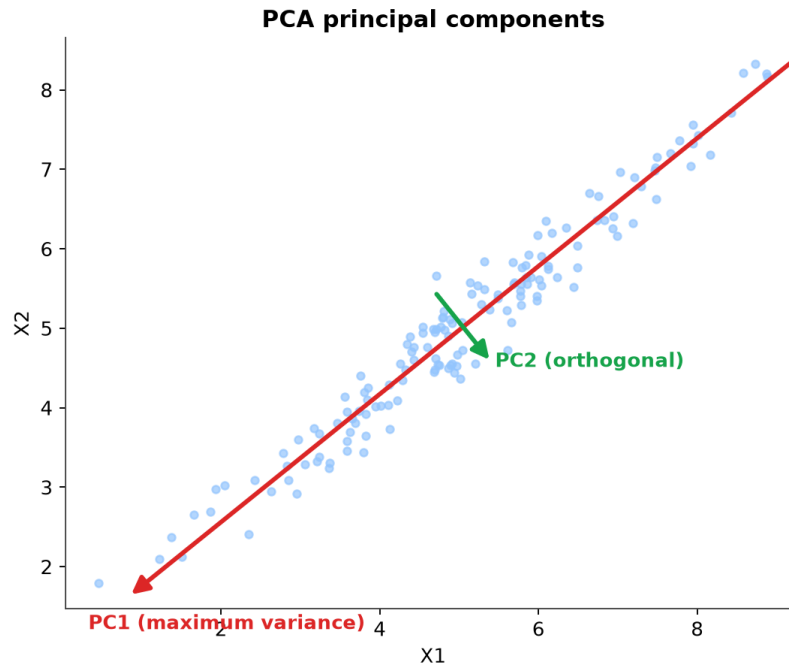


Figure3.12: Principal components: PC1 captures the maximum variance, PC2 is orthogonal.

Compression, reconstruction, and the linear autoencoder perspective

PCA can be conceptualized mathematically as a linear autoencoder. It consists of two main phases:

- **Encoder:** converts the input vector x_n into a reduced representation $z_n = B^T x_n$, where B contains the eigenvectors with the largest associated eigenvalues.
- **Decoder:** reconstructs the approximate projection back into the original space: $\hat{x}_n = B z_n$.

The average reconstruction error, or squared distortion, minimized by this procedure is exactly equal to the sum of the eigenvalues of the components that were discarded from the analysis.

A real example of PCA's usefulness appears in the *NCI60* dataset, where each sample has 6830 genes (dimensions). [Fig. 3.13](#) projects these samples onto their first three principal components: despite the huge original dimensionality, cell lines of the same cancer type (same color) tend to cluster together in this reduced space.

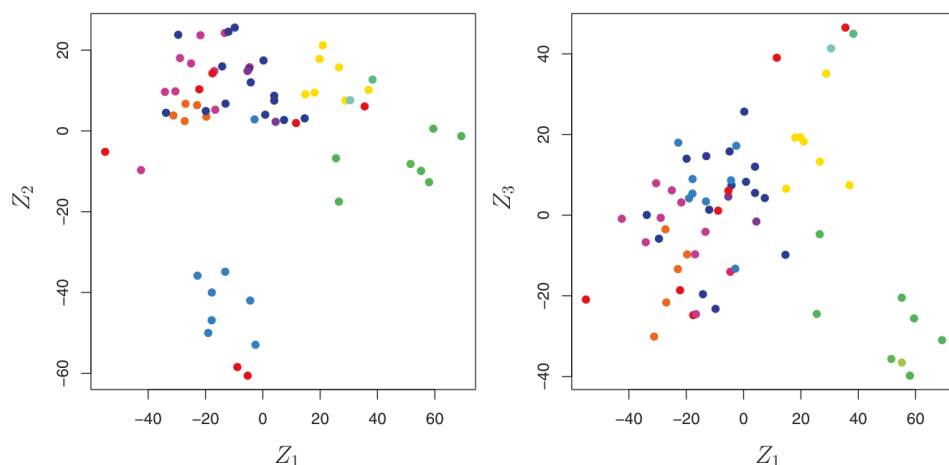


Figure 3.13: Projection of the *NCI60* cancer cell lines (6830 genes) onto their first three principal components. Source: James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*, Figure 10.15. Springer. Freely distributed for educational use (statlearning.com).

3.2.4 Non-linear methods and manifold learning

Although PCA is highly robust and fast, it suffers from an obvious limitation: it assumes that the interesting subspace of the data is linear (a flat hyperplane). When data relationships are intrinsically non-linear, as in the classic Swiss roll example (a curved 3D sheet of paper), PCA's linear orthogonal projection collapses distant points and artificially mixes information.

The manifold hypothesis

Manifold learning assumes the validity of the manifold hypothesis: all natural real-world data (MNIST images, faces, voices, or text) lie on a low-dimensional manifold embedded within the original high-dimensional space.

- A manifold is a continuous, curved surface that locally resembles a flat Euclidean linear space.
- For example, the space of handwritten digit images (28x28 pixels = 784 dimensions) is constrained by physical laws and invariances (stroke thickness, tilt, continuity) that greatly reduce the system's real degrees of freedom, confining viable samples to a continuous manifold of very low dimension.
- The goal of non-parametric manifold learning is to learn embedded coordinates for each point so that the manifold's internal topology and distances along its surface are faithfully represented.

Fig. 3.14 shows the classic Swiss roll example: the data (left) is rolled up in 3D, but its true structure is a 2D surface that can be “unrolled” (right) while preserving distances along the manifold.

Key non-linear algorithms

Kernel PCA (kPCA)

To extend PCA to non-linear problems, the kernel trick is used. kPCA implicitly maps the input data into an ultra-high (or infinite) dimensional Hilbert space $\Phi(x)$, where complex non-linear relationships become linearly separable or projectable.

- Using common kernel functions such as the RBF (Gaussian) kernel, complex projections can be modeled.

Swiss roll: the motivation for manifold learning

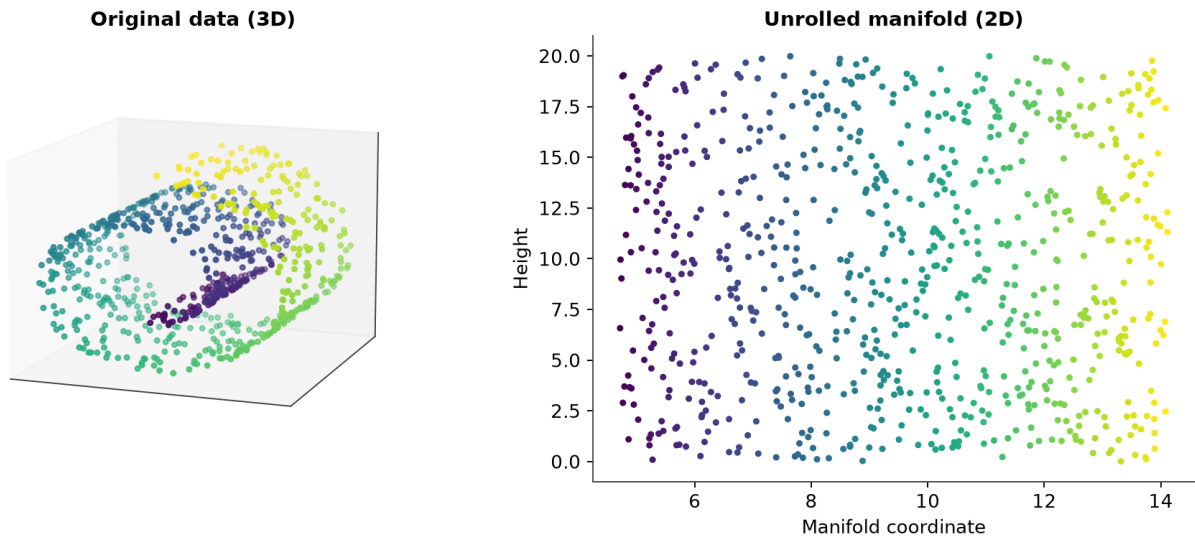


Figure 3.14: The Swiss roll: data rolled up in 3D (left) and its unrolled manifold in 2D (right).

- **Drawback:** unlike standard PCA, kPCA does not define a direct, invertible projection mapping for out-of-sample data, and poorly tuned kernels can sometimes expand and distort the space rather than usefully compressing it.

Fig. 3.15 illustrates the kernel trick with a classic example: two classes arranged in concentric circles (left) cannot be separated by a straight line in 2D. Applying the map $\phi(x_1, x_2) = (x_1, x_2, x_1^2 + x_2^2)$ (right) lifts the classes to different heights, where a simple horizontal plane separates them perfectly.

t-SNE (t-Distributed Stochastic Neighbor Embedding)

Proposed by Maaten and Hinton (2008) [vdMH08], t-SNE is the preferred non-convex technique for the qualitative visualization of complex groupings in two dimensions.

1. **High-dimensional space** (original SNE): converts Euclidean distances between samples into Gaussian conditional probabilities $p_{j|i}$ denoting similarity. Nearby points receive high neighborhood probabilities and distant ones receive infinitesimal probabilities.
2. **The crowding problem:** when high-dimensional data is projected onto a flat 2D space, the available volume of space shrinks exponentially. Average distances grow so much that, using normal approximations, attraction forces push all distant points into a dense, indistinguishable core at the center of the plot.
3. **The Student's t-distribution solution:** t-SNE solves this limitation by using a Student's t-distribution with one degree of freedom (equivalent to a Cauchy distribution) in the low-dimensional space. Its much heavier tails, inverted in the denominator of the latent probability equation q_{ij} , eliminate unwanted attraction forces between distant clusters:

$$q_{ij} = \frac{(1 + \|z_i - z_j\|^2)^{-1}}{\sum_{k \neq i} (1 + \|z_k - z_i\|^2)^{-1}}$$

The gradient acts much like a physical attraction-repulsion law (similar to forces between galaxies and stars), allowing clusters to organize and separate optimally in the visual plane.

Fig. 3.16 compares both methods on a classic example: two interleaved “moons” embedded in a 30-dimensional space. PCA (left), being a linear projection, can only rotate the data and fails to separate the two classes, which remain interleaved. t-SNE (right) rearranges the points non-linearly while preserving local neighborhoods, producing two clearly distinct groups.

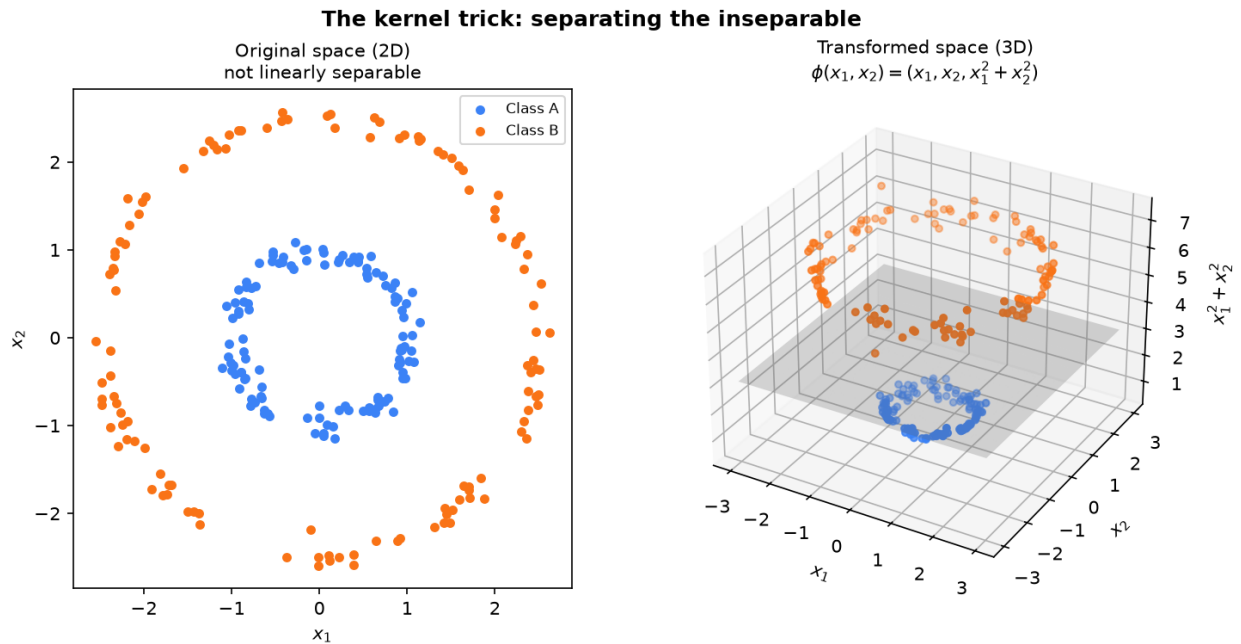


Figure3.15: The kernel trick: data that is not separable in 2D becomes separable once projected into an extra dimension.

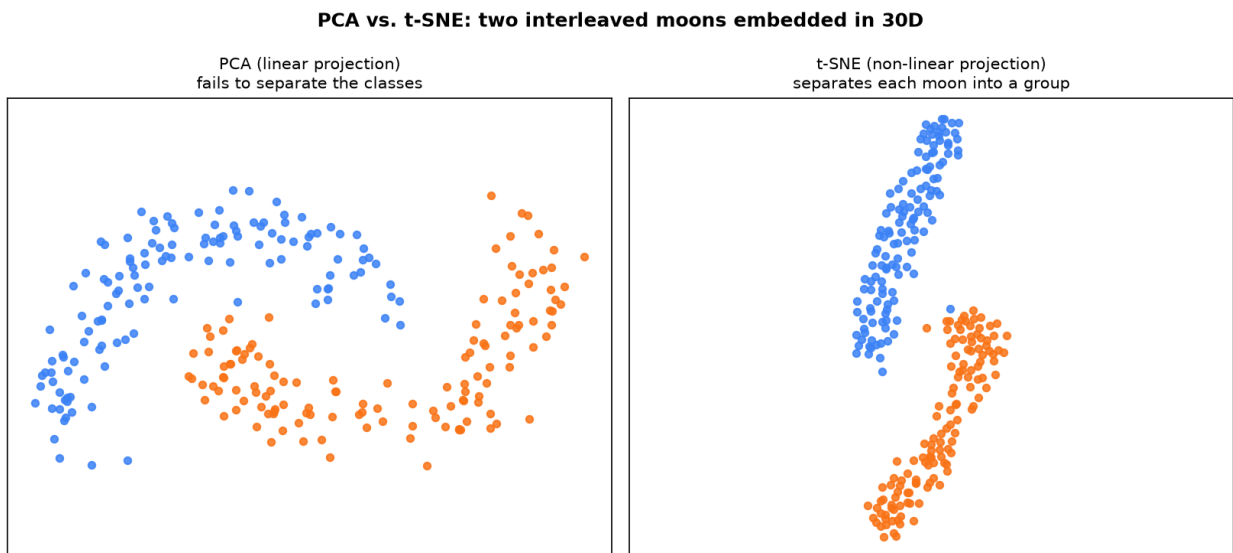


Figure3.16: PCA (linear projection) versus t-SNE (non-linear projection) on data with non-linear structure.

UMAP (Uniform Manifold Approximation and Projection)

UMAP [MHM18] is one of the most powerful manifold learning techniques available today. Grounded in classical Riemannian geometry and algebraic topology, UMAP assumes the data space is locally connected and that the manifold on which it lies is uniform.

Unlike t-SNE, which focuses almost exclusively on retaining very local neighborhoods (short-range relationships), UMAP is able to preserve both the local and the global structure of the data.

It is mathematically much more efficient, resulting in substantially faster execution speed on massive datasets with millions of samples.

Fig. 3.17 shows a 2D UMAP projection of the handwritten digits dataset (64 dimensions): each digit (0-9, one color per class) forms a compact, well-differentiated group, and the relative position between groups is also informative.

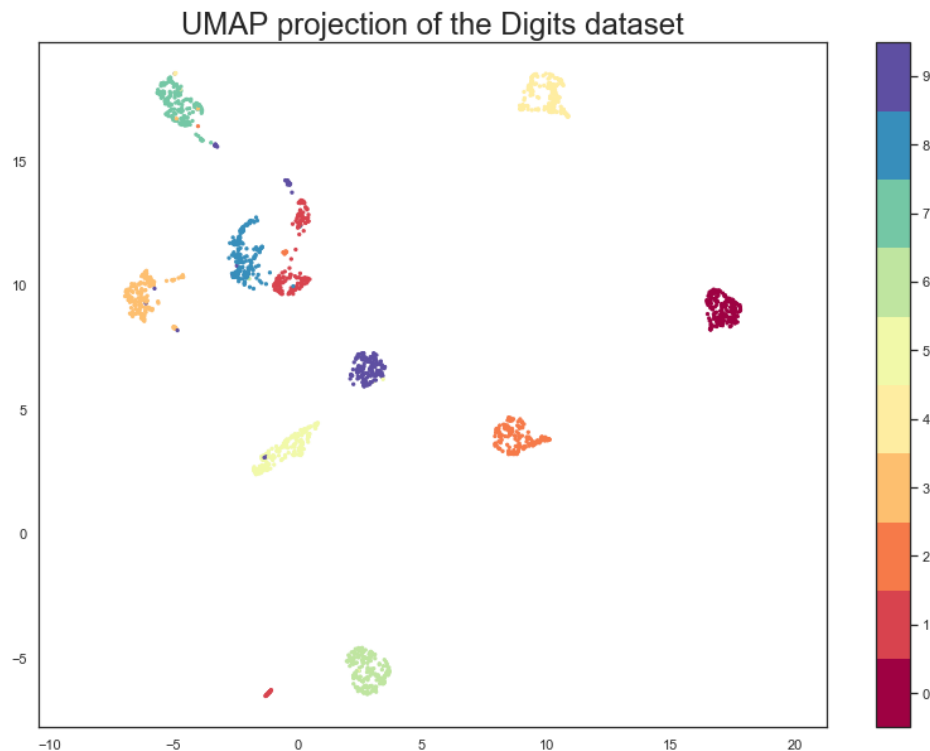


Figure 3.17: UMAP projection of the *Digits* dataset. Source: umap-learn documentation, “How to Use UMAP” (umap-learn.readthedocs.io). Copyright (c) 2017, Leland McInnes. BSD 3-Clause License.

3.2.5 Hands-on example in Java with SMILE

The `programacion-avanzada-smile` repository includes the full example `Ejemplo06PCA.java`, which applies PCA to the *Iris* dataset (four variables) with the `SMILE` library. The following fragment summarizes its main steps (comments translated into English).

```
import java.util.Arrays;
import smile.classification.KNN;
import smile.data.DataFrame;
import smile.feature.extraction.PCA;
import smile.io.Read;
import smile.tensor.Vector;
```

(continues on next page)

(continued from previous page)

```

import smile.validation.metric.Accuracy;

DataFrame iris = Read.csv("data/iris.csv", "header=true").factorize("species");
double[][] x = iris.drop("species").toArray();
int[] y = iris.column("species").toIntArray();

// 1. Fit PCA on the covariance matrix.
//    PCA.cor(x) would use the correlation matrix (equivalent to standardizing first).
PCA pca = PCA.fit(x);

// 2. Variance explained by each component (the "scree plot" as a table)
Vector proporcion = pca.varianceProportion();
Vector acumulada = pca.cumulativeVarianceProportion();
for (int i = 0; i < proporcion.size(); i++) {
    System.out.printf("PC%d: %.2f%% (cumulative %.2f%%)\n",
        i + 1, 100 * proporcion.get(i), 100 * acumulada.get(i));
}
System.out.println(pca.loadings()); // loadings: weight of each variable in each
    component

// 3. Projection from 4 dimensions down to 2
PCA proyeccion2D = pca.getProjection(2);
double[][] x2 = proyeccion2D.apply(x);
System.out.println("First projected sample: " + Arrays.toString(x2[0]));

// Alternative: ask for the components needed to retain 95% of the variance
int componentes95 = pca.getProjection(0.95).apply(x)[0].length;

// 4. PCA as a preprocessing step before a classifier.
//    PCA is fit ONLY on the training set and then applied to the test set:
//    fitting it on all the data would be information leakage.
Utiles.ParticionArrays particion = Utiles.split(x, y, 0.7);
PCA pcaTrain = PCA.fit(particion.xTrain()).getProjection(2);
double[][] xTrain2 = pcaTrain.apply(particion.xTrain());
double[][] xTest2 = pcaTrain.apply(particion.xTest());

KNN<double[]> knnCompleto = KNN.fit(particion.xTrain(), particion.yTrain(), 5);
KNN<double[]> knnReducido = KNN.fit(xTrain2, particion.yTrain(), 5);
System.out.printf("KNN with 4 variables    accuracy = %.2f%%\n",
    100.0 * Accuracy.of(particion.yTest(), knnCompleto.predict(particion.
    xTest())));
System.out.printf("KNN with 2 components  accuracy = %.2f%%\n",
    100.0 * Accuracy.of(particion.yTest(), knnReducido.predict(xTest2)));

```

A few things worth noticing when you run it:

- `varianceProportion()` and `cumulativeVarianceProportion()` are the numerical equivalent of the scree plot and help decide how many components to keep.
- `getProjection(2)` fixes the number of components, whereas `getProjection(0.95)` chooses it automatically from the amount of variance to retain.
- When PCA is used as preprocessing, fitting it on all the data (training and test) would be **information leakage**: PCA must be fit on the training set only.
- The full example also includes kernel PCA (KernelPCA with a Gaussian kernel) and probabilistic PCA; SMILE additionally offers *t*-SNE and UMAP in the `smile.manifold` package.

To run it from the root of the repository: `mvn exec:java -Dexec.mainClass=es.usal.smile.`

Ejemplo06PCA.

3.2.6 Summary

- **PCA**: fast, linear, interpretable original features.
 - **t-SNE**: excellent visualization, non-linear, slow for large data.
 - **UMAP**: a balance between PCA and t-SNE.
 - **Explained variance**: a metric for choosing the number of components.
 - **Curse of dimensionality**: the motivation for reduction.
 - **Applications**: compression, visualization, preprocessing.
-

Next: In 3.3 we will learn to detect anomalies, the final topic in unsupervised learning.

3.3 Anomaly Detection

3.3.1 Introduction

Anomaly detection is an unsupervised task of great industrial relevance, aimed at identifying unusual patterns or suspicious events within a continuous stream of data. It is widely applied in financial fraud detection, corporate network intrusion prevention, manufacturing quality control, and automated dataset cleaning.

3.3.2 The philosophy of anomaly detection

At a conceptual level, anomaly detection rests on the premise that normal behaviors (called inliers) are highly frequent and share consistent behavioral patterns, while anomalies (outliers) are rare events whose characteristics deviate significantly from the norm.

Anomaly detection vs. novelty detection

Although these terms are sometimes used interchangeably in practice, the specialized literature draws a critical methodological distinction based on the contamination of the training dataset:

- **Anomaly/outlier detection**: the algorithm is trained on a dataset that has not been cleaned and may therefore contain an unknown percentage of naturally infiltrated outliers or noisy samples. The goal is both to identify these outliers within the training set itself (data cleaning) and to correctly classify new incoming observations.
- **Novelty detection**: the algorithm is trained under the strict assumption that the training dataset is completely clean and free of anomalies. The goal here is to precisely define the boundary of known “normal” behavior in order to label any new or unknown pattern as a “novelty.”

The unsupervised approach

Anomaly detection is fundamentally addressed through unsupervised learning because, in real-world situations, it is not known in advance what type of anomaly may occur. Attempting to train a traditional supervised classifier (such as logistic regression) proves ineffective due to the severe class imbalance and because future anomalies may present entirely novel patterns the model never saw during training.

3.3.3 Statistical and reconstruction-based approaches

Before resorting to more complex models, there are two highly efficient classic approaches based on density estimation and the linear projection of the data. Fig. 3.18 summarizes the scheme shared by both: a model (density-based or projection-based) is fit on the normal data and a threshold is set; each new observation is scored against that threshold and, if it shows low density or a high reconstruction error, it is flagged as an anomaly.

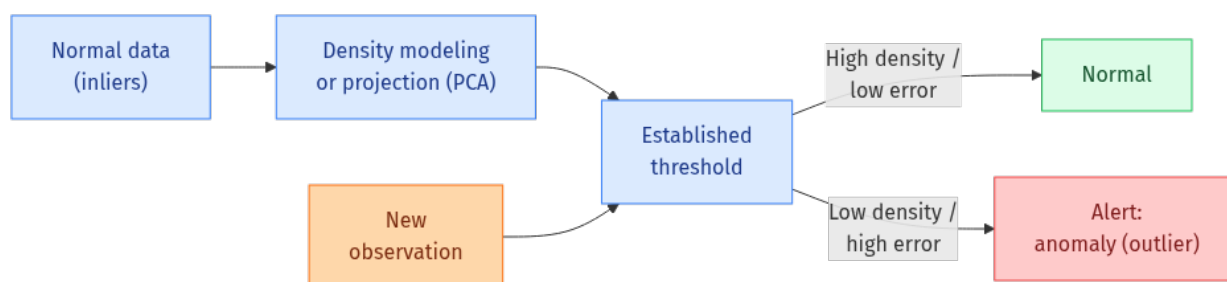


Figure 3.18: General scheme of the statistical and reconstruction-based approaches: model what is normal, set a threshold, and flag whatever falls outside.

Density modeling via Gaussian mixtures (GMM)

Under this approach, normal observations are assumed to concentrate in regions of the feature space with high probability density.

- A **Gaussian Mixture Model** (GMM) is trained to approximate the dataset's probability density function.
- To classify a new sample, its **density** under the estimated model is computed. Any instance located in a low-density region, below a preset threshold, is flagged as an anomaly.
- In real-world settings where the historical failure rate is known (for example, 4% defective products in a factory), the **threshold** is set mathematically by selecting the corresponding percentile (the 4% with lowest density under the model).

Fig. 3.19 illustrates the concept: the blue contours represent curves of equal probability density, and the red points marked with an X are anomalies falling outside the high-density regions.

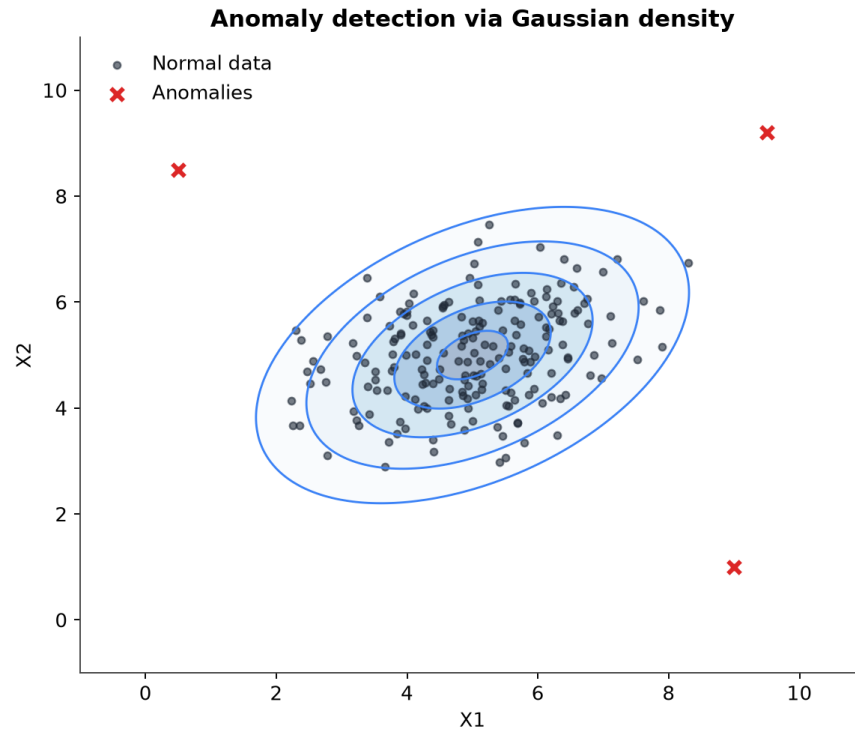


Figure 3.19: Anomaly detection via Gaussian density estimation.

Reconstruction approach using PCA

This technique is based on the principle that the majority principal components from a PCA analysis capture the directions of maximum variance that characterize the system's normal behavior.

- The dataset is projected into a low-dimensional space using PCA and subsequently reconstructed back into the original space using the inverse matrix.
- For each instance, the **reconstruction error** is computed (the squared distance between the original vector x and its reconstruction $\hat{x}$).
- Since the principal components do not capture the unusual deviations of outliers, anomalies will experience a significantly higher reconstruction error than normal instances, allowing them to be easily identified.

Fig. 3.20 illustrates the process: normal data (blue) are projected onto the PC1 line and reconstructed with minimal error (thin lines), while the two anomalies (red) lie far from the data's principal direction and, once reconstructed onto that same line, show a much larger reconstruction error (long arrows).

3.3.4 Specific unsupervised algorithms

Isolation forest

One of the most efficient and scalable algorithms [LTZ08] for outlier detection, especially designed to work in high-dimensional spaces.

- **Mechanics:** unlike traditional methods that try to model the density or the normal points, isolation forest explicitly seeks to isolate each observation. To do so, it builds a set of random decision trees. At each node of a tree, a feature is randomly selected, and a random cutoff threshold (between the minimum and maximum of that variable)

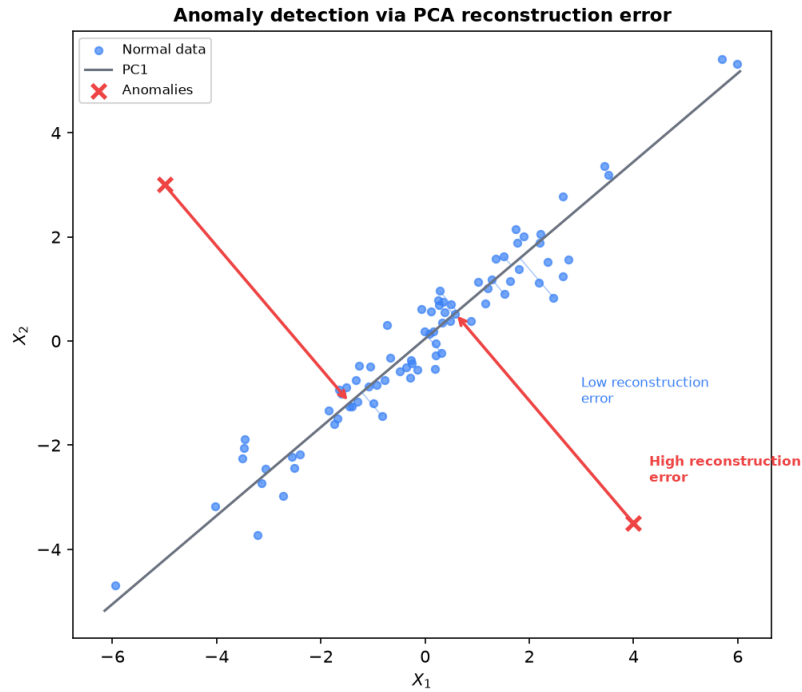


Figure 3.20: Anomaly detection via PCA reconstruction error.

is chosen to split the data in two. This recursive partitioning process continues until each instance is isolated in its own leaf.

- **Intuition:** since anomalies are far from the bulk of the normal data population, they require, on average, significantly fewer random partitions to be isolated. Therefore, instances with a shorter average path length to the root across the forest of trees are immediately flagged as anomalies.

Normal Data (Dense) —————> Requires many splits to isolate. Anomalies (Isolated/Rare) —————> Isolated quickly (few branches).

Fig. 3.21 compares both cases: the anomaly (left) is isolated with just 2 random splits, while a normal point (right) requires many more splits to be separated from the rest.

Local Outlier Factor (LOF)

This algorithm [BKNS00] bases its operation on analyzing the local density of samples using a k-nearest-neighbors (KNN) approach.

- LOF compares the **local density** of an instance with the density of its nearest neighbors.
- A normal instance will have a local density similar to that of its surroundings. In contrast, a **local outlier** will exhibit a significantly lower density than its nearest neighbors (it will be more isolated relative to the density of its immediate neighborhood).
- This method is extremely useful for detecting **local anomalies** that would not stand out in a global analysis because their absolute values are not extreme, yet they are unusual for the specific context of the cluster they belong to.

Fig. 3.22 shows a typical case: the point marked in red is not far from all the data in absolute terms, but its local density is much lower than that of its immediate neighbors, which gives it away as a local anomaly.

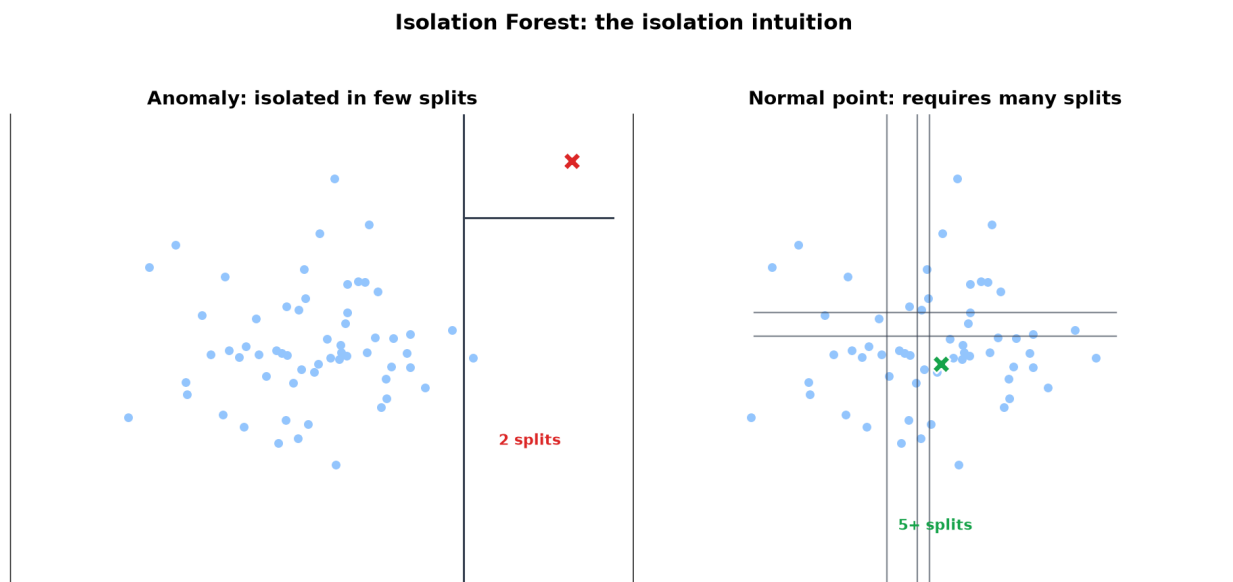


Figure3.21: Isolation Forest: anomalies are isolated in fewer partitions than normal points.

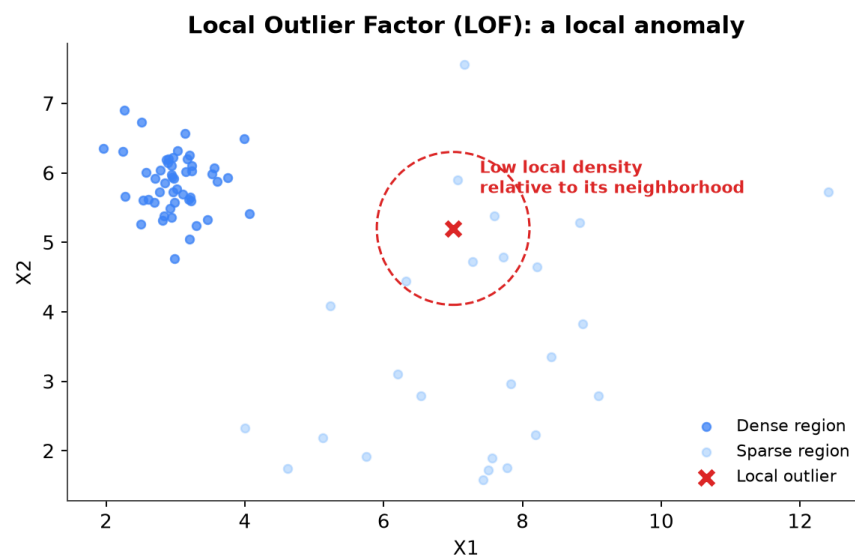


Figure3.22: Local Outlier Factor: a local anomaly does not stand out in a global analysis.

One-class SVM

This algorithm [ScholkopfPST+01] is specifically optimized for novelty detection in scenarios where a clean training dataset is available.

- **How it works:** instead of finding a hyperplane that separates two classes, one-class SVM projects the data into a high-dimensional feature space via a kernel and seeks to separate the training instances from the origin.
- **Decision boundary:** this is geometrically equivalent to finding the minimum-volume region or hypersphere that encloses nearly all of the training samples. If a new observation falls outside this region, bounded by the border support vectors, it is automatically classified as an anomaly or novelty.

Fig. 3.23 shows an example with irregularly shaped data: one-class SVM traces a non-linear boundary (purple) that tightly wraps around the region of normal data (blue), so that any new observation falling outside it (red crosses) is classified as an anomaly.

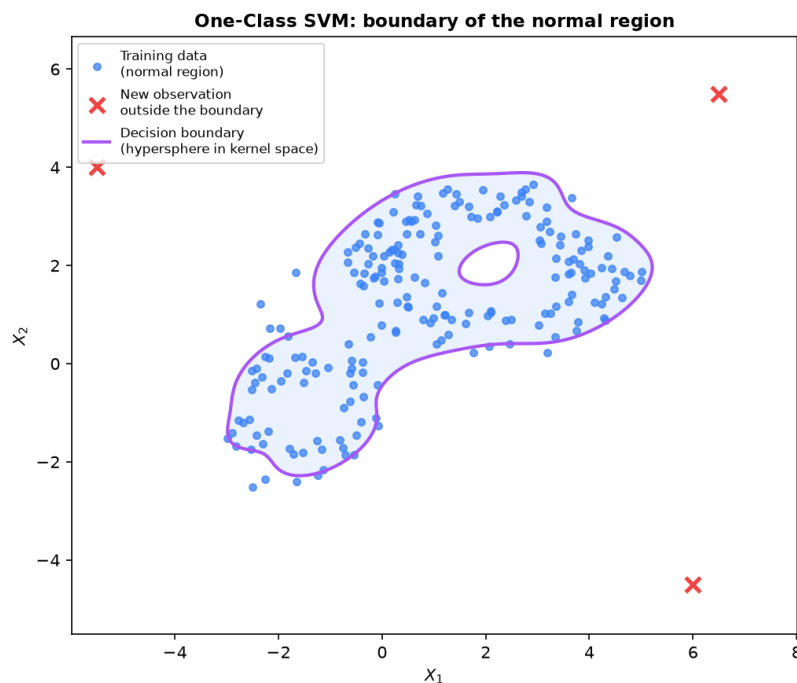


Figure3.23: One-class SVM: the boundary wraps the normal region; anything outside it is classified as an anomaly.

3.3.5 Summary

- **Anomalies:** rare, contextual, or collective events.
- **Simple methods:** Gaussian for 1D, PCA-based approaches.
- **Advanced methods:** Isolation Forest, Local Outlier Factor, one-class SVM.

Congratulations! You have completed the content of “Introduction to Machine Learning.” You now have a solid foundation to explore more advanced topics such as **deep learning**, **natural language processing**, or **computer vision**.

Bibliography

BIBLIOGRAPHY

This page lists the textbooks this course draws on, as well as the original articles and studies cited throughout the chapters.

Course reference books

- [Cho25] François Chollet. *Deep Learning with Python*. Manning Publications, Shelter Island, NY, third edition, 2025.
- [DFO20] Marc Peter Deisenroth, A. Aldo Faisal, and Cheng Soon Ong. *Mathematics for Machine Learning*. Cambridge University Press, Cambridge, 2020. Open-access edition available at mml-book.github.io.
- [Geron22] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. O'Reilly Media, Sebastopol, CA, third edition, 2022. ISBN 9781492032649.
- [HTF09] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, New York, second edition, 2009. Corrected 12th printing (2017); freely distributed textbook (statlearning.com). doi:[10.1007/978-0-387-84858-7](https://doi.org/10.1007/978-0-387-84858-7).
- [JWHT13] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. Springer, New York, 2013. Freely distributed textbook for educational use (statlearning.com). doi:[10.1007/978-1-4614-7138-7](https://doi.org/10.1007/978-1-4614-7138-7).
- [Mur22] Kevin P. Murphy. *Probabilistic Machine Learning: An Introduction*. MIT Press, Cambridge, MA, 2022. Open-access edition available at probml.ai; CC BY-NC-ND license.
- [WFH11] Ian H. Witten, Eibe Frank, and Mark A. Hall. *Data Mining: Practical Machine Learning Tools and Techniques*. Morgan Kaufmann, Burlington, MA, third edition, 2011. ISBN 9780123748560.

Cited articles and studies

- [Bre01] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi:[10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324).
- [BKNS00] Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jörg Sander. Lof: identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, 93–104, 2000. doi:[10.1145/342009.335388](https://doi.org/10.1145/342009.335388).
- [CV95] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995. doi:[10.1007/BF00994018](https://doi.org/10.1007/BF00994018).
- [EKSX96] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, 226–231, 1996.
- [Fis36] Ronald A. Fisher. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188, 1936. doi:[10.1111/j.1469-1809.1936.tb02137.x](https://doi.org/10.1111/j.1469-1809.1936.tb02137.x).
- [Fri01] Jerome H. Friedman. Greedy function approximation: a gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. doi:[10.1214/aos/1013203451](https://doi.org/10.1214/aos/1013203451).
- [Gau09] Carl Friedrich Gauss. *Theoria motus corporum coelestium in sectionibus conicis solem ambientium*. Friedrich Perthes und I.H. Besser, Hamburg, 1809.

- [HK70] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970. doi:10.1080/00401706.1970.10488634.
- [Hot33] Harold Hotelling. Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6):417–441, 1933. doi:10.1037/h0071325.
- [KSH12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25, 1097–1105. 2012.
- [Leg05] Adrien-Marie Legendre. *Nouvelles méthodes pour la détermination des orbites des comètes*. Firmin Didot, Paris, 1805. Historical origin of the least-squares method.
- [LTZ08] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining*, 413–422. 2008. doi:10.1109/ICDM.2008.17.
- [Mac67] James MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, 281–297. 1967.
- [MMRS06] John McCarthy, Marvin L. Minsky, Nathaniel Rochester, and Claude E. Shannon. A proposal for the dartmouth summer research project on artificial intelligence. *AI Magazine*, 27(4):12–14, 2006. Original proposal written in 1955; reprinted in *AI Magazine* in 2006. doi:10.1609/aimag.v27i4.1904.
- [MP43] Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4):115–133, 1943. doi:10.1007/BF02478259.
- [MHM18] Leland McInnes, John Healy, and James Melville. Umap: uniform manifold approximation and projection for dimension reduction. 2018. URL: <https://arxiv.org/abs/1802.03426>, arXiv:1802.03426.
- [Pea01] Karl Pearson. On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11):559–572, 1901. doi:10.1080/14786440109462720.
- [Ros58] Frank Rosenblatt. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958. doi:10.1037/h0042519.
- [Rou87] Peter J. Rousseeuw. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987. doi:10.1016/0377-0427(87)90125-7.
- [RHW86] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi:10.1038/323533a0.
- [ScholkopfPST+01] Bernhard Schölkopf, John C. Platt, John Shawe-Taylor, Alex J. Smola, and Robert C. Williamson. Estimating the support of a high-dimensional distribution. *Neural Computation*, 13(7):1443–1471, 2001. doi:10.1162/089976601750264965.
- [Tib96] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, 58(1):267–288, 1996. doi:10.1111/j.2517-6161.1996.tb02080.x.
- [TWH01] Robert Tibshirani, Guenther Walther, and Trevor Hastie. Estimating the number of clusters in a data set via the gap statistic. *Journal of the Royal Statistical Society: Series B*, 63(2):411–423, 2001. doi:10.1111/1467-9868.00293.
- [Tur50] Alan M. Turing. Computing machinery and intelligence. *Mind*, 59(236):433–460, 1950. doi:10.1093/mind/LIX.236.433.
- [vdMH08] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9:2579–2605, 2008.
- [VSP+17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. 2017.

About this book

ABOUT

This book contains the material for the **Fundamentals and Introduction to Machine Learning** block of the Advanced Programming course in the Computer Engineering degree at the University of Salamanca (USAL).

Citation information, BibTeX, DOI and Zenodo metadata are collected on the [How to Cite](#) page.

How this book is made

This website is written in Markdown files and Jupyter notebooks, and converted to HTML and PDF using tools from TeachBooks, Jupyter Book and MyST Markdown.

The source files are stored in a public GitHub repository:

- <https://github.com/dmjimenezbravo/ml-teachbook-usal>

The published version of the book can be viewed at:

- <https://dmjimenezbravo.github.io/ml-teachbook-usal/>

About the author



Diego M. Jiménez Bravo

Permanent Lecturer, Computer Science and Automation

- USAL profile: <https://produccioncientifica.usal.es/investigadores/148175/detalle?lang=en>
- ORCID: <https://orcid.org/0000-0002-0291-7627>

Institution

Faculty of Sciences, University of Salamanca (USAL).

Photographs and academic profile information come from the University of Salamanca scientific production portal.

CREDITS AND LICENSES

Project credits

This book and the course template are built with [TeachBooks](#), on top of the [Jupyter Book](#) and [MyST Markdown](#) ecosystem.

TeachBooks provides documentation, components and working patterns that make it easier to create open, interactive and reproducible teaching books.

Acknowledgements

We thank the [TeachBooks](#) project and [Delft University of Technology \(TU Delft\)](#) for their work on open tools for educational books. The [credits page in the TeachBooks Manual](#) is used as a reference for explicitly acknowledging the tools, communities and institutional support on which this material builds.

TeachBooks, Jupyter Book and MyST Markdown are acknowledged here as tools, documentation and technical communities on which this project builds. They are not part of the book authorship or of the creator metadata declared in `CITATION.cff` and Zenodo.

Book license

This book is distributed under the [Creative Commons Attribution 4.0 International \(CC BY 4.0\)](#) license.

This allows the material to be shared and adapted, as long as attribution is provided.

External resources

Unless explicitly stated on a specific page, the content of this book is authored by the project authors.

If any external resources are reused in a page, they should be acknowledged on that page together with their corresponding attribution and license.

Code and tools

This project uses external tools and libraries, including:

- [TeachBooks](#)
- [Jupyter Book](#)
- [MyST Markdown](#)

Each tool keeps its own license.

HOW TO CITE

Zenodo DOI: <https://doi.org/10.5281/zenodo.20449102>

If you use this material, please cite it as follows.

Recommended Citation (APA 7th edition)

Jiménez-Bravo, D. M. (2026). *Fundamentos e introducción al aprendizaje automático* [Fundamentals and introduction to machine learning]. Universidad de Salamanca. <https://doi.org/10.5281/zenodo.20449102>

BibTeX

You can also use the downloadable BibTeX file: `citation.bib`.

```
@book{jimenez_bravo_fundamentos_2026,  
  author = {Jiménez-Bravo, Diego M.},  
  title = {Fundamentos e introducción al aprendizaje automático},  
  year = {2026},  
  publisher = {Universidad de Salamanca},  
  doi = {10.5281/zenodo.20449102},  
  url = {https://doi.org/10.5281/zenodo.20449102},  
  note = {Source code: https://github.com/dmjimenezbravo/ml-teachbook-usal}  
}
```

Author

- Diego M. Jiménez Bravo, ORCID: <https://orcid.org/0000-0002-0291-7627>

DOI

Final Zenodo DOI for this version: <https://doi.org/10.5281/zenodo.20449102>.